Fortran 77 Tutorial

Erik Boman, Sarah T. Whitlock, Paul H. Hargrove

Adapted and extended by Roberto Augusto Gómez Loenzo (2026)

Copyright © 1995-7 by Stanford University

New material © 2026 Roberto Augusto Gómez Loenzo,
licensed under CC BY-NC-SA 4.0
(see “A note on this edition” in the preface)

Contents

1 Preface **9**
A note on this edition 9

2 What is Fortran? **11**
2.1 Why learn Fortran 77? 12
2.2 Portability 12
2.3 Exercises 12

3 Fortran 77 Basics **15**
3.1 Program organization 16
3.2 Column position rules 17
3.3 Comments 17
3.4 Continuation 18
3.5 Blank spaces 19
3.6 Putting it all together 19
3.7 Exercises 19

4 How to use Fortran with gfortran **21**
4.1 Introduction to gfortran 21
4.1.1 Installing gfortran 21
4.2 Source code, object code, compiling, and linking 22
4.2.1 Basic compilation 22
4.2.2 Compilation process in detail 23
4.2.3 Example: Compiling and running a simple program . 23
4.3 Compiling multiple files 24
4.4 Useful compiler options 24
4.5 Building with Make 25
4.6 Debugging Fortran programs 25
4.7 The programs of this tutorial 26
4.8 Resources for learning more about gfortran 26
4.9 Exercises 27

5 Variables, types, and declarations **29**
5.1 Variable names . . . 29
5.2 Types and declarations . . . 29
5.3 Integers and floating point variables . . . 31
5.4 The `parameter` statement . . . 32
5.5 The `implicit` statement . . . 34
5.6 Type Conversion . . . 35
5.7 The `equivalence` statement . . . 36
5.7.1 Restrictions on `equivalence` . . . 36
5.7.2 Example: Looking at the bits of a real number . . . 37
5.7.3 Cautions about `equivalence` . . . 37
5.8 Character Variables and String Operations . . . 38
5.9 Exercises . . . 38

6 Expressions and assignment **41**
6.1 Constants . . . 41
6.2 Expressions . . . 42
6.3 Assignment . . . 43
6.4 Type conversion . . . 43
6.5 Character strings and string manipulation . . . 45
6.5.1 Character declaration . . . 45
6.5.2 Character arrays . . . 45
6.5.3 Character assignment . . . 45
6.5.4 String operations . . . 46
6.5.5 String intrinsic functions . . . 46
6.5.6 Substring operations . . . 47
6.5.7 Example: String processing . . . 48
6.6 Exercises . . . 49

7 Logical expressions **51**
7.1 Logical variables and assignment . . . 51

8 The `if` statements **53**
8.1 Nested `if` statements . . . 54
8.2 Exercises . . . 54

9 Loops **55**
9.1 do-loops . . . 55
9.1.1 How many times is the loop executed? . . . 56
9.2 while-loops . . . 57
9.3 until-loops . . . 58
9.4 Advanced control statements . . . 58
9.4.1 Arithmetic `if` statement . . . 59
9.4.2 Unconditional `goto` . . . 59

9.4.3 Computed `goto` . . . 59
9.4.4 Assigned `goto` . . . 61
9.4.5 Alternate `return` . . . 61
9.4.6 Non-block `do` . . . 62
9.4.7 Best practices . . . 63
9.5 Exercises . . . 63

10 Arrays **65**
10.1 One-dimensional arrays . . . 65
10.2 Two-dimensional arrays . . . 66
10.3 Storage format for 2-dimensional arrays . . . 67
10.4 Multi-dimensional arrays . . . 67
10.5 The `dimension` statement . . . 68
10.6 Exercises . . . 68

11 Subprograms **69**
11.1 Statement functions . . . 69
11.2 Functions . . . 70
11.2.1 Why the function must be declared . . . 72
11.3 Subroutines . . . 73
11.4 Call-by-reference . . . 74
11.4.1 A dangerous mistake: passing a constant . . . 75
11.5 The `SAVE` statement . . . 75
11.5.1 When to use `SAVE` . . . 77
11.5.2 Restrictions on `SAVE` . . . 77
11.5.3 Best practices with `SAVE` . . . 78
11.6 Exercises . . . 78

12 Arrays in subprograms **81**
12.1 Variable length arrays . . . 81
12.2 Passing subsections of arrays . . . 82
12.3 Different dimensions . . . 83
12.4 Exercises . . . 86

13 Common blocks and include files **87**
13.1 Common blocks example . . . 87
13.2 Common blocks syntax . . . 88
13.3 Arrays in common blocks . . . 90
13.4 Blank common blocks . . . 91
13.5 The `INCLUDE` statement . . . 92
13.5.1 Benefits of `INCLUDE` . . . 93
13.5.2 Limitations of `INCLUDE` . . . 93
13.5.3 Best practices for `INCLUDE` files . . . 94
13.6 Exercises . . . 94

14 Data and Block Data Statements **97**
14.1 The data statement 97
14.2 The block data statement 98
14.3 Exercises 99

15 File I/O **101**
15.1 Opening and closing a file 101
15.1.1 Basic File Opening and Closing Example 102
15.2 Read and write revisited 103
15.3 Error Handling 103
15.3.1 Using IOSTAT for Error Detection 103
15.3.2 Using ERR for Error Handling 104
15.4 File STATUS Parameter 104
15.5 File ACCESS Methods 105
15.5.1 Sequential Access 105
15.5.2 Direct Access 106
15.6 File FORM Parameter 107
15.7 End-of-File Detection 107
15.8 Comprehensive Error Handling Example 108
15.9 File Existence Checking 109
15.10Appending to Files 110
15.11A complete example: points in space 110
15.12Exercises 112

16 Simple I/O **115**
16.1 Read and write 115
16.2 Examples 115
16.3 Other versions 116
16.4 Implied do-loops 117
16.5 Exercises 118

17 Format statements **119**
17.1 Syntax 119
17.2 Common format codes 120
17.3 Some examples 120
17.4 Format strings in read/write statements 121
17.5 Implicit loops and repeat counts 122
17.6 Internal files 123
17.7 Putting it together: a table of circle areas 124
17.8 NAMELIST I/O 125
17.8.1 Defining a NAMELIST group 125
17.8.2 Reading with NAMELIST 125
17.8.3 Writing with NAMELIST 127
17.8.4 Advantages of NAMELIST I/O 127

17.8.5 Example: Parameter input with NAMELIST 127
17.8.6 Limitations of NAMELIST I/O 129
17.9 Exercises 129

18 Numerical Software **131**
18.1 Netlib 131
18.2 Other sources of numerical software 132
18.3 BLAS 133
18.3.1 Levels and naming conventions 133
18.3.2 BLAS 1 133
18.3.3 BLAS 2 134
18.3.4 BLAS 3 134
18.3.5 Examples 135
18.3.6 How to get the BLAS 136
18.3.7 Documentation 137
18.4 LAPACK 138
18.4.1 Routines 138
18.4.2 Documentation 138
18.4.3 Example: solving a linear system 140
18.4.4 How to get LAPACK 141
18.5 Using libraries 142
18.5.1 Creating your own library 142
18.6 Searching for mathematical software 144
18.7 Exercises 144

19 Fortran programming style **147**
19.1 Portability 147
19.2 Program structure 147
19.3 Comments 148
19.4 Indentation 148
19.5 Variables 148
19.6 Subprograms 148
19.7 Goto 148
19.8 Arrays 149
19.9 Efficiency concerns 149
19.10Exercises 149

20 Debugging hints **151**
20.1 Useful compiler options 151
20.2 Some common errors 153
20.3 Debugging tools 153
20.4 Exercises 154

21 Fortran resources on the Web **155**
21.1 The language 155
21.2 Compilers and tools 155
21.3 Numerical software 156
21.4 The original tutorial 156

Chapter 1

Preface

The goal of this Fortran tutorial is to give a quick introduction to the most common features of the Fortran 77 programming language. A companion tutorial introduces the key enhancements found in Fortran 90. It is *not* a complete reference! Many details have been omitted. The presentation focuses on scientific computations, mainly linear algebra. The outline of this tutorial was inspired by the book *"Handbook for Matrix Computations"* by T.F. Coleman and C. Van Loan, published by SIAM. Sections of this tutorial pertaining to Fortran 90 were based on the book *"FORTRAN 77 for Engineers and Scientists with an Introduction to Fortran 90"* by L. Nyhoff and S. Leestma published by Prentice Hall.

This tutorial was designed by Erik Boman to be used in the course *SCCM-001-F: Introduction to Fortran* taught at Stanford University, Winter quarter 1996. It has been modified by Sarah T. Whitlock and Paul H. Hargrove for use in the Fortran courses which have been offered under different course numbers each subsequent year. Permission to use this tutorial for educational and other non-commercial purposes is granted provided all author and copyright information is retained.

Paul H. Hargrove, Stanford, December 1997.

Sarah T. Whitlock, Stanford, January 1997.

Erik Boman, Stanford, December 1995.

Copyright © 1995-7 by Stanford University. All rights reserved.

A note on this edition

This is an adapted and extended edition of the Fortran 77 tutorial written at Stanford University by Erik Boman, Sarah T. Whitlock and Paul H. Hargrove, whose original preface appears above, unchanged. The original tutorial is still available at `https://web.stanford.edu/class/me200c/`

tutorial_77/. This edition was prepared by Roberto Augusto Gómez Loenzo in 2026 and is distributed free of charge, for educational and other non-commercial purposes, under the permission granted in the original preface.

The main changes with respect to the original are:

- corrections of technical errors, and updates of the parts that had become outdated (compilers, tools, the status of obsolescent and deleted features in the later Fortran standards, and the resources on the Web);
- a new chapter on compiling with gfortran, and new or extended sections in several chapters (for example statement functions, `save`, internal files, implied do-loops, error handling in file I/O, and debugging options);
- exercises at the end of every chapter (the original tutorial had none);
- the directory `code` that accompanies this edition, with all the example programs completed so that they can be compiled and run, the tested solutions of the exercises, and a script that compiles and runs all of them.

The references above to a companion Fortran 90 tutorial and to sections on Fortran 90 belong to the original edition; they are not part of this one.

This edition is *not* an official version of the Stanford tutorial, and it has not been reviewed or endorsed by its authors or by Stanford University. Any errors introduced in it are the responsibility of the adapter.

The parts taken from the original tutorial remain Copyright © 1995-7 by Stanford University, under the terms of its original permission. The new material of this edition (new and rewritten text, exercises and their solutions, and the programs in the `code` directory) is Copyright © 2026 Roberto Augusto Gómez Loenzo and is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0), https://creativecommons.org/licenses/by-nc-sa/4.0/. In short, you may copy and adapt it for non-commercial purposes, as long as you credit both the original authors and the adapter, and distribute your adaptations under the same terms.

Chapter 2

What is Fortran?

Fortran is a general purpose programming language, mainly intended for mathematical computations in science and engineering. Fortran is an acronym for FORmula TRANslation, and was originally capitalized as FORTRAN. However, following the current trend to only capitalize the first letter in acronyms, we will call it Fortran. Fortran was the first high-level programming language to be widely used. The work on Fortran started in the 1950's at IBM, under the direction of John Backus, and the first compiler was delivered in 1957. There have been many versions since. By convention, a Fortran version is denoted by the last two digits of the year the standard was proposed (or, since 2003, by all four digits). Thus we have

- FORTRAN 66 (the first standard)
- FORTRAN 77
- Fortran 90 and its minor revision Fortran 95
- Fortran 2003, Fortran 2008, Fortran 2018 and Fortran 2023

Fortran 90 was a major revision: it introduced free-form source code, array operations, dynamic memory allocation, modules, derived types (records) and much more. The later standards added, among other things, object-oriented programming, interoperability with the C language, and parallel programming with *coarrays*. All of these standards are almost completely backwards compatible: with very few exceptions, a standard-conforming Fortran 77 program is also a valid program in the later standards, and today's compilers compile it without problems.

Users should be aware that most Fortran 77 compilers allow a superset of Fortran 77, i.e. they allow non-standard extensions. In this tutorial we will emphasize standard ANSI Fortran 77, and we will point out the extensions we use.

2.1 Why learn Fortran 77?

Fortran is still widely used in engineering and science, especially for computationally intensive work such as weather and climate models, fluid dynamics, structural analysis and computational chemistry, and it is one of the main languages used on supercomputers. A huge amount of this software was written in Fortran 77, or in later versions of Fortran in a style that is very close to it. Classic numerical libraries such as BLAS and LAPACK (see Chapter 18) are still distributed as Fortran 77 code. It is therefore important for engineers and scientists to be able to read, use and modify Fortran 77 code.

From time to time, so-called experts predict that Fortran will rapidly fade in popularity and soon become extinct. These predictions have always failed. Fortran is the most enduring computer programming language in history. One of the reasons is *software inertia*: once a company or a research group has spent many years of work on a piece of software, it is unlikely to translate it to a different language. Reliable software translation is a very difficult task. Another reason is that Fortran compilers produce very fast code for numerical computations.

If you are going to write a *new* program, you should consider using the features of modern Fortran (Fortran 90 and later). But everything you learn in this tutorial remains valid, and it is the best preparation for working with the large body of existing Fortran code.

2.2 Portability

A major advantage Fortran has is that it is standardized by ANSI and ISO (see footnotes). Consequently, if your program is written in ANSI Fortran 77, using nothing outside the standard, then it will run on any computer that has a Fortran 77 compiler. Thus, Fortran programs are portable across machine platforms. The complete text of the Fortran 77 standard is available on the Web site of WG5, the ISO committee responsible for Fortran: `https://wg5-fortran.org/ARCHIVE/Fortran77.html`.

Footnotes:
ANSI = American National Standards Institute
ISO = International Organization for Standardization

2.3 Exercises

Answers are in the file `code/exercises/ch02/answers.md` that accompanies this tutorial.

1. What does the name Fortran mean? Who directed its development, and in which year was the first compiler delivered?

2. List the versions of Fortran in chronological order, and name three important new features of Fortran 90.

3. Explain in your own words what *software inertia* is, and why it helps to keep Fortran 77 in use.

4. Open the text of the Fortran 77 standard at `https://wg5-fortran.org/ARCHIVE/Fortran77.html` and read its Section 1.1. According to that section, what is the purpose of the standard? Relate it to the idea of portability.

Chapter 3

Fortran 77 Basics

A Fortran program is just a sequence of lines of text. The text has to follow a certain *structure* to be a valid Fortran program. Let's start with the simplest possible program:

```
      program hello
c This is a simple "Hello, World!" program in Fortran 77

      write (*,*) 'Hello, World!'

      stop
      end
```

This program contains the minimum elements required in every Fortran program: a program declaration, executable statements (in this case, just one output statement), and an end marker. It simply displays the text "Hello, World!" to the screen.

Now let's look at a slightly more complex example:

```
      program circle
      real r, area

c This program reads a real number r and prints
c the area of a circle with radius r.

      write (*,*) 'Give radius r:'
      read  (*,*) r
      area = 3.14159*r*r
      write (*,*) 'Area = ', area

      stop
      end
```

The lines that begin with with a "c" are *comments* and have no purpose other than to make the program more readable for humans. Originally, all Fortran programs had to be written in all upper-case letters. Most people now write lower-case since this is more legible, and so will we. You may

wish to mix case, but Fortran is not case-sensitive, so "X" and "x" are the same variable.

Here's another example showing basic arithmetic operations:

```
      program arithmetic
      integer a, b, sum, diff, prod
      real quotient

c This program demonstrates basic arithmetic operations

      a = 15
      b = 4

      sum = a + b
      diff = a - b
      prod = a * b
      quotient = real(a) / real(b)

      write (*,*) 'a = ', a, ' b = ', b
      write (*,*) 'Sum: ', sum
      write (*,*) 'Difference: ', diff
      write (*,*) 'Product: ', prod
      write (*,*) 'Quotient: ', quotient

      stop
      end
```

Notice in this example how we convert integers to real values before division to ensure we get a decimal result rather than truncated integer division.

3.1 Program organization

A Fortran program generally consists of a main program (or driver) and possibly several subprograms (procedures or subroutines). For now we will place all the statements in the main program; subprograms will be treated later. The structure of a main program is:

```
      program name

      declarations

      statements

      stop
      end
```

In this tutorial, words that are in *italics* should not be taken as literal text, but rather as a description of what belongs in their place.

The `stop` statement is optional and may seem superfluous since the program will stop when it reaches the end anyway, but it is recommended

to always terminate a program with the `stop` statement to emphasize that the execution flow stops there.

You should note that you cannot have a variable with the same name as the program.

Here's a practical example showing a temperature conversion program:

```
      program temperature
      real fahr, celsius

c This program converts Fahrenheit to Celsius

      write (*,*) 'Enter temperature in Fahrenheit:'
      read (*,*) fahr

      celsius = (fahr - 32.0) * 5.0 / 9.0

      write (*,*) fahr, ' degrees Fahrenheit equals'
      write (*,*) celsius, ' degrees Celsius'

      stop
      end
```

3.2 Column position rules

Fortran 77 is *not* a free-format language, but has a very strict set of rules for how the source code should be formatted. The most important rules are the column position rules:

```
Col. 1    : Blank, or a "c" or "*" for comments
Col. 1-5  : Statement label (optional)
Col. 6    : Continuation of previous line (optional)
Col. 7-72 : Statements
Col. 73-80: Sequence number (optional, rarely used today)
```

Most lines in a Fortran 77 program starts with 6 blanks and ends before column 72, i.e. only the statement field is used.

3.3 Comments

A line that begins with the letter "c" or an asterisk in the first column is a comment. Comments may appear anywhere in the program. Well-written comments are crucial to program readability. Commercial Fortran codes often contain about 50% comments. You may also encounter Fortran programs that use the exclamation mark (!) for comments. This is not a standard part of Fortran 77, but is supported by several Fortran 77 compilers and is explicitly allowed in Fortran 90. When understood, the exclamation mark may appear anywhere on a line (except in positions 2-6).

Here's an example demonstrating different comment styles:

```
      program comments
c Traditional Fortran 77 comment with 'c' in column 1
* Alternate Fortran 77 comment with '*' in column 1

      write (*,*) 'This program demonstrates comment styles'
c     write (*,*) 'This line is commented out'
* The next line shows inline comments (non-standard but commonly
    supported)
      write (*,*) 'Hello' ! This is an inline comment

      stop
      end
```

3.4 Continuation

Sometimes, a statement does not fit into the 66 available columns of a single line. One can then break the statement into two or more lines, and use the continuation mark in position 6. Example:

```
c23456789 (This demonstrates column position!)

c The next statement goes over two physical lines
      area = 3.14159265358979
     +       * r * r
```

Any character can be used instead of the plus sign as a continuation character. It is considered good programming style to use either the plus sign, an ampersand, or digits (using 2 for the second line, 3 for the third, and so on).

Here's a more practical example of using continuation lines for a complex calculation:

```
      program long_equation
      real x, y, result

c This program calculates a complex formula using continuation lines

      x = 2.5
      y = 3.7

      result = 3.14159265358979 * x**2 + 2.71828182846 * y**2
     +         + 1.4142135623 * x * y + (x + y) / 2.0
     +         - (x - y)**2

      write (*,*) 'x = ', x, ' y = ', y
      write (*,*) 'Result = ', result

      stop
      end
```

3.5 Blank spaces

Blank spaces are ignored in Fortran 77. So if you remove all blanks in a Fortran 77 program, the program is still acceptable to a complier but almost unreadable to humans.

3.6 Putting it all together

Let's conclude with a final example that combines several of the concepts we've covered. This program calculates both the area and perimeter of a rectangle:

```
      program rectangle
      real length, width, area, perimeter

c This program computes the area and perimeter of a rectangle

      write (*,*) 'Enter rectangle length:'
      read (*,*) length
      write (*,*) 'Enter rectangle width:'
      read (*,*) width

      area = length * width
      perimeter = 2.0 * (length + width)

      write (*,*) 'Rectangle dimensions: ', length, ' x ', width
      write (*,*) 'Area: ', area
      write (*,*) 'Perimeter: ', perimeter

      stop
      end
```

This example demonstrates input/output operations, variable declarations, comments, and arithmetic calculations—all fundamental aspects of Fortran 77 programming.

3.7 Exercises

Solutions are in the directory `code/exercises/ch03` that accompanies this tutorial.

1. Write a program that reads a temperature in degrees Celsius and prints it in degrees Fahrenheit, using $F = \frac{9}{5}C + 32$. Test it with 100 degrees (the answer is 212). Be careful to write `9.0/5.0` and not `9/5`; Chapter 6 explains why.

2. Write a program that reads three numbers and prints their sum and their average.

3. The following program does not compile, because some of its lines break the column rules. Find the errors and correct them.

```
1       program errors
2        real base, height
3        base = 2.0
4     height = 3.0
5  * Area of a triangle
6        write (*,*) 'Area of the triangle:',
7        0.5*base*height
8        end
```

4. Write a program that reads the base and the height of a triangle and prints its area. Start the program with a comment that explains what it does, and split the statement that computes the area over two lines with a continuation line.

5. Type in the following program, taking care that the number 25 starts exactly in column 72 (the comment line is a ruler that shows the last digit of each column number). Compile it, run it, and explain the result. Then compile it again with the option `-Wall`.

```
1       program trunc
2       real a, b, total
3       a = 1.0
4       b = 2.0
5  c234567890123456789012345678901234567890123456789012345678901234567890123456789012
6       total = a + b +                                                          25
7       write (*,*) 'total =', total
8       end
```

Chapter 4

How to use Fortran with gfortran

4.1 Introduction to gfortran

In this chapter, we'll learn how to compile and run Fortran 77 programs using gfortran, which is the Fortran compiler from the GNU Compiler Collection (GCC). gfortran is freely available, widely used, and runs on most operating systems including Linux, macOS, and Windows.

4.1.1 Installing gfortran

Before you can compile Fortran programs, you'll need to have gfortran installed on your system.

- **On Linux:** Most Linux distributions provide gfortran through their package managers. For example:
  - Ubuntu/Debian: `sudo apt install gfortran`
  - Fedora: `sudo dnf install gcc-gfortran`
  - Arch Linux: `sudo pacman -S gcc-fortran`
- **On macOS:** You can install gfortran using Homebrew:
  - Install Homebrew first (if you don't have it), following the instructions at https://brew.sh
  - Then install gfortran: `brew install gcc`
- **On Windows:** You have several options:
  - Install MSYS2 and then, in its UCRT64 terminal, the package with gfortran:
    `pacman -S mingw-w64-ucrt-x86_64-gcc-fortran`

  - Use Windows Subsystem for Linux (WSL) and install gfortran there as on Linux
  - Install Cygwin with its gfortran package

To verify that gfortran is installed correctly, open a terminal (or Command Prompt on Windows) and type:

```
gfortran --version
```

You should see output showing the version of gfortran installed on your system.

4.2 Source code, object code, compiling, and linking

A Fortran program consists of plain text that follows certain rules (syntax). This is called the *source code*. You need to use a text editor to write (edit) the source code. There are many text editors available, such as:

- Visual Studio Code with the Modern Fortran extension
- Sublime Text with appropriate syntax highlighting
- Vim or Emacs with Fortran syntax support
- Notepad++ on Windows

Modern text editors provide useful features like syntax highlighting, which makes it easier to identify different parts of your code.

When you have written a Fortran program, you should save it in a file with the extension *.f* or *.for* for Fortran 77 code (or *.f90* for more modern Fortran). Before you can execute the program, you need to translate it into machine-readable form. This is done by a program called a *compiler*. gfortran will convert your source code into an executable program.

4.2.1 Basic compilation

The simplest way to compile a Fortran program with gfortran is:

```
gfortran program.f -o program
```

Where:

- `program.f` is your source file
- `-o program` specifies the name of the executable file to create

If you omit the `-o` option, gfortran will create an executable named `a.out` on Linux/macOS or `a.exe` on Windows.

To run the program, in your terminal or command prompt:

```
# On Linux/macOS
./program

# On Windows
program.exe
```

4.2.2 Compilation process in detail

The compilation process actually involves several steps:

1. **Preprocessing:** Handles any preprocessing directives (not common in Fortran 77)
2. **Compilation:** Converts the source code to object code
3. **Linking:** Combines object files and libraries to create an executable

You can perform these steps separately if needed:

```
# Compile to object file only
gfortran -c program.f

# This creates program.o (an object file)
# Then link to create executable
gfortran program.o -o program
```

4.2.3 Example: Compiling and running a simple program

Let's compile and run the circle area program `circle` from Chapter 3. We will meet this program several times in this tutorial: each time we learn something new, we will use it to improve the program a little.

First, save the program to a file named `circle.f`. Then, compile and run it:

```
# Compile
gfortran circle.f -o circle

# Run
./circle
```

When you run the program, it will prompt you to enter a radius. After you enter a value (say 2) and press Enter, it will calculate and display the area of the circle:

```
 Give radius r:
2
 Area =    12.5663605
```

Notice the extra blank at the beginning of each output line and the wide field used for the number. This is how *list-directed* output (`write (*,*)`) looks; in Chapter 17 we will learn how to control the layout.

4.3 Compiling multiple files

For larger projects, your code might be split across multiple files. For example, suppose you have a main program in `main.f` and some subroutines in `subs.f`. You can compile them together:

```
gfortran main.f subs.f -o program
```

Or compile them separately and then link:

```
gfortran -c main.f
gfortran -c subs.f
gfortran main.o subs.o -o program
```

This second approach is useful for larger projects, as you only need to recompile files that have changed.

4.4 Useful compiler options

gfortran provides many compiler options that can help detect errors and improve your code:

`-Wall` Enable all common warning messages. This helps catch potential problems.

`-Wextra` Enable additional warnings beyond those in `-Wall`.

`-std=legacy` Accept old (legacy) Fortran code, including obsolete features, without complaining. This is useful for compiling old programs; note that it is the *most permissive* mode, not a strict one. gfortran has no option that checks for strict Fortran 77; the closest is `-std=f95`, which reports the features that were deleted or are non-standard in Fortran 95.

`-fimplicit-none` Require explicit declarations of all variables (similar to adding `implicit none` to your code).

`-g` Include debugging information in the executable, useful when using a debugger.

`-O2` Optimize code for speed (level 2 optimization).

`-fcheck=bounds` Check for array bounds violations at runtime (slows down execution but helps find bugs). The older spelling `-fbounds-check` also works. `-fcheck=all` turns on all run-time checks.

Example with multiple options:

```
gfortran -Wall -fcheck=all -g circle.f -o circle
```

4.5 Building with Make

For complex projects with many files, it's cumbersome to manually compile each file. The `make` utility automates this process with a Makefile that specifies dependencies between files.

Here's a simple example of a Makefile for a project with two source files:

```
# Simple Makefile for Fortran 77 project

# Compiler and flags
FC = gfortran
FFLAGS = -Wall -g

# Target executable
TARGET = myprogram

# Object files
OBJS = main.o subs.o

# Default rule
all: $(TARGET)

# Rule to create the executable
$(TARGET): $(OBJS)
        $(FC) -o $@ $(OBJS)

# Rule for compiling Fortran source files
%.o: %.f
        $(FC) $(FFLAGS) -c $<

# Clean rule
clean:
        rm -f $(OBJS) $(TARGET)
```

With this Makefile in your project directory, you can simply run:

```
make
```

And it will compile only the files that have changed since the last build.

4.6 Debugging Fortran programs

When your program doesn't work as expected, you can use a debugger to help find the issue. The GNU Debugger (GDB) works with gfortran. To use it:

1. Compile your program with the `-g` flag to include debugging information:

   ```
   gfortran -g program.f -o program
   ```

2. Start the debugger:

   ```
   gdb ./program
   ```

3. Set breakpoints, run the program, and examine variables. Some basic GDB commands:

   - `break 10` - Set a breakpoint at line 10
   - `run` - Start the program
   - `print x` - Print value of variable x
   - `step` - Execute next line, stepping into subroutines
   - `next` - Execute next line, stepping over subroutines
   - `continue` - Continue execution until next breakpoint
   - `quit` - Exit GDB

Many editors and development environments offer a graphical interface to gdb, which may be more user-friendly; for example, Visual Studio Code with the Modern Fortran extension and a debugging extension for gdb. Chapter 20 gives more hints on debugging.

4.7 The programs of this tutorial

All the programs in this tutorial, the code fragments turned into complete programs, and the solutions to the exercises are available in the directory `code` that accompanies this tutorial, organized by chapter (`code/ch05`, `code/ch06`, ...). The script `run_tests.sh` in that directory compiles and runs all of them, so you can easily check that they work with your compiler. We encourage you to type in at least the first few programs yourself, though: making (and fixing) small mistakes is part of learning a language.

4.8 Resources for learning more about gfortran

To learn more about gfortran and its capabilities, here are some useful resources:

- The official gfortran documentation: https://gcc.gnu.org/onlinedocs/gfortran/

- GNU Fortran Wiki: https://gcc.gnu.org/wiki/GFortran
- Fortran Programming Wikibook: https://en.wikibooks.org/wiki/Fortran

Remember that gfortran supports both Fortran 77 and modern Fortran standards. As you gain experience, you might want to explore newer Fortran features available in Fortran 90, 95, 2003, 2008, and beyond.

4.9 Exercises

The files mentioned below, and the answers, are in the directory `code/exercises/ch04` that accompanies this tutorial.

1. Install gfortran and check the installation with `gfortran -version`. Then compile and run the programs `hello.f` and `circle.f` of Chapter 3.
2. Compile `circle.f` in two steps: first only to an object file, then link it. Which files exist after each step?
3. The program in `main.f` uses a function that is in another file, `funcs.f` (functions are explained in Chapter 11). Compile both files into one executable, first with a single command and then in separate steps. Then write a Makefile for them, and check that after modifying only `funcs.f`, `make` does not recompile `main.f`.
4. The program `typo.f` has a misspelled variable name. Compile and run it without options, then with `-Wall`, and finally with `-fimplicit-none`. Explain what happens in each case.

Chapter 5

Variables, types, and declarations

5.1 Variable names

Variable names in Fortran consist of 1-6 characters chosen from the letters `a-z` and the digits `0-9`. The first character must be a letter. Fortran 77 does not distinguish between upper and lower case, in fact, it assumes all input is upper case. However, nearly all Fortran 77 compilers will accept lower case. If you should ever encounter a Fortran 77 compiler that insists on upper case it is usually easy to convert the source code to all upper case.

Strictly speaking, some of the names used in this tutorial (for example `variables_demo` or `r_sum`) break this rule: they are longer than six characters or contain an underscore. Fortran 90 allows names of up to 31 characters (63 since Fortran 2003) including underscores, and practically every Fortran 77 compiler in use today, gfortran included, accepts them. If you need to write strictly standard Fortran 77, keep to six letters and digits.

Unlike most programming languages, Fortran has no *reserved words*. Words such as `program`, `real`, `stop` and `end` have a special meaning, but the compiler recognizes them from their position in a statement, so it is legal (though very confusing!) to have a variable called `real` or `end`. Do not take advantage of this: choose names that cannot be confused with the words of the language.

5.2 Types and declarations

Every variable *should* be defined in a *declaration*. This establishes the *type* of the variable. The most common declarations are:

```
1       integer   list of variables
2       real      list of variables
3       double precision  list of variables
```

```
      complex   list of variables
      logical   list of variables
      character list of variables
```

The *list of variables* should consist of variable names separated by commas. Each variable should be declared exactly once. If a variable is undeclared, Fortran 77 uses a set of *implicit rules* to establish the type. This means all variables starting with the letters `i-n` are integers and all others are real. Many old Fortran 77 programs uses these implicit rules, but *you should not!* The probability of errors in your program grows dramatically if you do not consistently declare your variables.

```
      program variables_demo
      integer i, count, sum
      real x, y, average
      double precision bignum, smallnum
      logical flag1, flag2
      character*10 firstname
      character*15 lastname

c This program demonstrates variable declarations

      i = 10
      count = 25
      sum = i + count

      x = 3.14159
      y = 2.71828
      average = (x + y) / 2.0

      bignum = 1.23456789D+15
      smallnum = 9.87654321D-12

      flag1 = .TRUE.
      flag2 = .FALSE.

      firstname = 'John'
      lastname = 'Smith'

      write (*,*) 'Integer values:'
      write (*,*) '  i =', i, '  count =', count, '  sum =', sum
      write (*,*) 'Real values:'
      write (*,*) '  x =', x, '  y =', y, '  average =', average
      write (*,*) 'Double precision values:'
      write (*,*) '  bignum =', bignum
      write (*,*) '  smallnum =', smallnum
      write (*,*) 'Logical values:'
      write (*,*) '  flag1 =', flag1, '  flag2 =', flag2
      write (*,*) 'Character values:'
      write (*,*) '  firstname =', firstname
      write (*,*) '  lastname =', lastname

      stop
```

```
42        end
```

5.3 Integers and floating point variables

Fortran 77 has only one type for integer variables. Integers are usually stored as 32 bits (4 bytes) variables. Therefore, all integer variables should take on values in the range [-m,m] where m is approximately 2×10^9.

```
1         program integer_range
2         integer small, big, result
3
4   c This program demonstrates integer ranges and overflow
5
6         small = 10
7         big = 2000000000  ! Close to maximum 32-bit integer
8
9         write (*,*) 'small =', small
10        write (*,*) 'big =', big
11
12        result = big + big  ! This will likely overflow
13
14        write (*,*) 'big + big =', result
15        write (*,*) 'This result may be incorrect due to integer overflow'
16
17        stop
18        end
```

Output with gfortran:

```
small =          10
big =  2000000000
big + big =  -294967296
This result may be incorrect due to integer overflow
```

The correct sum, 4 000 000 000, does not fit in 32 bits: the largest integer is $2^{31} - 1 = 2\,147\,483\,647$. On most machines the result simply "wraps around": $4\,000\,000\,000 - 2^{32} = -294\,967\,296$. No error message is given, so the program happily goes on with a negative number. The Fortran standard says nothing about what happens on overflow, so you must never rely on this behaviour; make sure your integers stay well inside the allowed range.

Fortran 77 has two different types for floating point variables, called `real` and `double precision`. While `real` is often adequate, some numerical calculations need very high precision and `double precision` should be used. Usually a real is a 4 byte variable and the double precision is 8 bytes, but this is machine dependent. Some non-standard Fortran versions use the syntax `real*8` to denote 8 byte floating point variables.

```
1         program precision_demo
2         real r_sum, r_val
3         double precision d_sum, d_val
```

```
      integer i

c This program demonstrates precision differences

      r_val = 0.1
      d_val = 0.1D0

      r_sum = 0.0
      d_sum = 0.0D0

c Add 0.1 ten times
      do 10 i = 1, 10
         r_sum = r_sum + r_val
         d_sum = d_sum + d_val
10    continue

      write (*,*) 'Expected sum: 1.0'
      write (*,*) 'Sum using REAL:', r_sum
      write (*,*) 'Sum using DOUBLE PRECISION:', d_sum
      write (*,*) 'Difference from 1.0 (REAL):', r_sum - 1.0
      write (*,*) 'Difference from 1.0 (DOUBLE):', d_sum - 1.0D0

      stop
      end
```

Output with gfortran:

```
 Expected sum: 1.0
 Sum using REAL:   1.00000012
 Sum using DOUBLE PRECISION:  0.99999999999999989
 Difference from 1.0 (REAL):   1.19209290E-07
 Difference from 1.0 (DOUBLE):  -1.1102230246251565E-016
```

Neither sum is exactly 1. The reason is that computers store numbers in binary, and 0.1 has no exact binary representation (just as 1/3 has no exact decimal representation), so a tiny error is made each time. A `real` keeps about 7 significant decimal digits, a `double precision` about 16. The lesson: never test floating point numbers for exact equality. Instead of `if (x .EQ. 1.0)` write something like `if (abs(x - 1.0) .LT. 1.0E-5)`.

5.4 The `parameter` statement

Some constants appear many times in a program. It is then often desirable to define them only once, in the beginning of the program. This is what the `parameter` statement is for. It also makes programs more readable. For example, the circle area program should rather have been written like this:

```
      program circle
      real r, area, pi
      parameter (pi = 3.14159)

c This program reads a real number r and prints
```

```
c the area of a circle with radius r.

      write (*,*) 'Give radius r:'
      read  (*,*) r
      area = pi*r*r
      write (*,*) 'Area = ', area

      stop
      end
```

The syntax of the parameter statement is

```
      parameter (name = constant, ... , name = constant)
```

The rules for the `parameter` statement are:

- The *name* defined in the `parameter` statement is not a variable but rather a constant. (You cannot change its value at a later point in the program.)
- A *name* can appear in at most one `parameter` statement.
- The `parameter` statement(s) must come before the first executable statement.

Some good reasons to use the `parameter` statement are:

- It helps reduce the number of typos.
- It makes it easier to change a constant that appears many times in a program.
- It increases the readability of your program.

```
      program physics_constants
      double precision pi, e, c, h, g
      parameter (pi = 3.14159265358979D0,
     +           e = 2.71828182845905D0,
     +           c = 299792458.0D0,      ! Speed of light (m/s)
     +           h = 6.62607015D-34,     ! Planck's constant (Jůs)
     +           g = 9.80665D0)          ! Gravity (m/s^2)

c This program demonstrates using multiple parameters
c for scientific constants

      write (*,*) 'Mathematical constants:'
      write (*,*) '  pi =', pi
      write (*,*) '  e =', e
      write (*,*) 'Physical constants:'
      write (*,*) '  Speed of light =', c, 'm/s'
      write (*,*) '  Planck constant =', h, 'Jůs'
      write (*,*) '  Gravity =', g, 'm/s^2'
```

```
19
20 c Cannot change a parameter value:
21 c     pi = 3.14   ! This would cause a compilation error
22
23       stop
24       end
```

5.5 The `implicit` statement

By default, Fortran uses the implicit typing rule that variables starting with the letters I through N are integer, while all other variables are real. This can be a source of subtle bugs. A much safer approach is to use the `implicit none` statement, which requires all variables to be explicitly declared:

```
1       implicit none
```

This statement must appear before any variable declarations and before the first executable statement. With `implicit none`, the compiler will generate an error for any undeclared variable.

While `implicit none` is not part of the ANSI Fortran 77 standard, it is supported by practically all compilers (and it is standard since Fortran 90), and it is highly recommended. **From now on, all complete programs in this tutorial start with `implicit none`.** With gfortran you can get the same effect without changing the source code by compiling with the option `-fimplicit-none`.

In strict Fortran 77, you can change the implicit typing rules using the standard `implicit` statement. Each of the following lines is an independent example; a letter may receive only one type, so they cannot all appear in the same program unit:

```
1 c Make every variable double precision:
2       implicit double precision (a-z)
3
4 c Keep the default rule, but make names starting with d double:
5       implicit double precision (d)
```

The first form is common in old numerical codes as `implicit double precision (a-h, o-z)`: names starting with `i`–`n` remain integer, all others become double precision. However, it's still better practice to explicitly declare all variables.

```
1       program implicit_demo
2       implicit none
3
4 c All variables must be declared when using implicit none
5       integer i, j
6       real x, y
7
8 c Without declarations, these would cause compiler errors:
```

```
c     k = 10
c     z = 3.14

      i = 5
      j = 10
      x = 2.5
      y = 3.5

      write (*,*) i*j, x*y

      stop
      end
```

5.6 Type Conversion

Fortran provides several built-in functions for converting between different data types:

```
      program type_conversion
      implicit none
      integer i, j
      real x, y

c This program demonstrates type conversion

      i = 5
      j = 2

      write (*,*) 'Integer division:', i/j
      write (*,*) 'Real division:', real(i)/real(j)

      x = 3.7
      y = 2.2

      write (*,*) 'Real values: x =', x, ' y =', y
      write (*,*) 'Truncated to integers:', int(x), int(y)
      write (*,*) 'Rounded to integers:',
     +             nint(x), nint(y)

      write (*,*) 'Character to integer:'
      write (*,*) '  Code for ''A'' =', ichar('A')
      write (*,*) '  Code for ''a'' =', ichar('a')
      write (*,*) 'Integer to character:'
      write (*,*) '  Character with code 66 =', char(66)
      write (*,*) '  Character with code 98 =', char(98)

      stop
      end
```

5.7 The `equivalence` statement

The `equivalence` statement allows two or more variables or array elements to share the same memory location. This can be used to save memory (though this is rarely a concern with modern computers) or to create different views of the same data. The syntax is:

```
1       equivalence (list1), (list2), ..., (listN)
```

where each *list* is a comma-separated list of variables or array elements that will share the same memory location.

Here's a simple example:

```
1       real a, b
2       integer i, j
3       equivalence (a, i), (b, j)
```

In this example, `a` and `i` will share the same memory location, as will `b` and `j`. Changing the value of `a` will also change `i`, and vice versa. Similarly for `b` and `j`.

The `equivalence` statement can also be used with arrays:

```
1       real arr1(100), arr2(50)
2       equivalence (arr1(1), arr2(1))
```

In this case, the first 50 elements of `arr1` will overlap with `arr2`.

It is important to understand what `equivalence` does *not* do: it never converts a value. The two names simply give two different interpretations of the same bits in memory. For example, equivalencing the integer 20230415 with a `character*8` variable does *not* give you the text `'20230415'`, but a meaningless sequence of bytes (and standard Fortran 77 does not even allow a character variable to be equivalenced with a numeric one). To convert numbers to text and back, use *internal files*, described in Section 17.6.

5.7.1 Restrictions on `equivalence`

There are several important restrictions on the use of `equivalence`:

- Character variables may only be equivalenced with other character variables (lengths may differ).
- Two variables that are both in common blocks cannot be equivalenced to each other. A local variable may be equivalenced with a variable in a common block, but then it becomes part of that block, and this must not extend the block backwards (before its first element).
- An equivalence must not require a variable to occupy two different places in memory, e.g. `equivalence (a(1), b(1)), (a(2), b(3))`.
- Dummy arguments of subprograms cannot be equivalenced.

5.7.2 Example: Looking at the bits of a real number

Since `equivalence` gives two views of the same memory, it can be used to look at how a real number is actually stored. The following program shows the bit pattern of a real number, interpreted as an integer:

```
1       program bitpattern
2       implicit none
3       real x
4       integer ix
5       equivalence (x, ix)
6
7       write(*,*) 'Enter a real number:'
8       read(*,*) x
9       write(*,*) 'Value as real:', x
10      write(*,*) 'Binary pattern as integer:', ix
11
12      stop
13      end
```

If you enter 1, the output with gfortran is:

```
 Enter a real number:
1
 Value as real:   1.00000000
 Binary pattern as integer:  1065353216
```

The number 1065353216 is $3F800000$ in hexadecimal, i.e. the bits `0 01111111 00000000000000000000000`: a sign bit (0, positive), an 8-bit exponent (127, which means 2^0) and a 23-bit fraction (0). This is the IEEE 754 format used by practically all current computers. On a machine with a different floating point format, the integer would be different: programs like this one are not portable.

5.7.3 Cautions about `equivalence`

While `equivalence` can be useful in some situations, it should be used with caution:

- It can make code harder to understand and maintain.
- It can introduce subtle bugs, especially when mixing different data types.
- It can interfere with compiler optimizations.
- Results may vary across different compilers or hardware platforms.
- For converting between numbers and text, use internal files (Section 17.6). Fortran 90 also offers the intrinsic function `transfer`, which copies the bits of one type into another without the need for `equivalence`.

5.8 Character Variables and String Operations

```
      program string_demo
      implicit none
      character*10 first_name
      character*15 last_name
      character*30 full_name

c This program demonstrates character variables

      first_name = 'John'
      last_name = 'Smith'

c Concatenate strings
      full_name = first_name // ' ' // last_name

      write (*,*) 'First name: [', first_name, ']'
      write (*,*) 'Last name: [', last_name, ']'
      write (*,*) 'Full name: [', full_name, ']'

c Character strings are fixed length and padded with spaces
      write (*,*) 'Length of first_name:', len(first_name)
      write (*,*) 'Length of full_name:', len(full_name)

      stop
      end
```

5.9 Exercises

Solutions are in the directory `code/exercises/ch05` that accompanies this tutorial (programs `ex1.f`, `ex2.f`, ..., and explanations in `answers.md`).

1. A day has 86400 seconds. Write a program that reads a number of days and prints the corresponding number of seconds, using integer variables. Try 1000 and 30000 days. Why is the second answer wrong? What is the largest number of days for which the program gives the right answer?

2. Write a program that adds 0.1 one million times, once in a `real` variable and once in a `double precision` variable, and compare both sums with the exact result 100000. Compare with the output of `precision_demo` above: why is the error so much larger now?

3. Write a program that reads the radius of a circle and prints its circumference and its area. Define π only once, with a `parameter` statement.

4. Which of the following are valid variable names in strict Fortran 77? Which of them does gfortran accept? `x1`, `1x`, `total`, `average`, `temp_c`, `real`, `my name`, `I`.

5. Without compiling it, predict what this program prints. Then compile it and check. (It has neither `implicit none` nor declarations, on purpose.)

```
      program implic
      n = 3.7
      x = 7/2
      y = 7.0/2.0
      write (*,*) 'n =', n
      write (*,*) 'x =', x
      write (*,*) 'y =', y
      end
```

6. Write a program that computes the average of the three integers 7, 8 and 8, first with `(a + b + c) / 3` and then correctly. Explain the difference.

7. Write a program that prints `int(x)` and `nint(x)` for $x = 2.7$, -2.7, 2.5 and -2.5. What is the rule of each function?

8. Run the program `bitpattern` with the numbers 2.0 and -1.0, and explain the results with the IEEE format described above.

9. A variable is declared as `character*5` and another as `character*10`. What do they contain after assigning `'Fortran'` to each of them? Check your answer with a program.

Chapter 6

Expressions and assignment

6.1 Constants

The simplest form of an expression is a *constant*. There are 6 types of constants, corresponding to the 6 data types. Here are some integer constants:

```
1
0
-100
32767
+15
```

Then we have real constants:

```
1.0
-0.25
2.0E6
3.333E-1
```

The E-notation means that you should multiply the constant by 10 raised to the power following the "E". Hence, 2.0E6 is two million, while 3.333E-1 is approximately one third.

For constants that are larger than the largest real allowed, or that requires high precision, double precision should be used. The notation is the same as for real constants except the "E" is replaced by a "D". Examples:

```
2.0D-1
1D99
```

Here 2.0D-1 is a double precision one-fifth, while 1D99 is a one followed by 99 zeros.

The next type is complex constants. This is designated by a pair of constants (integer or real), separated by a comma and enclosed in parentheses. Examples are:

```
(2, -3)
(1., 9.9E-1)
```

The first number denotes the real part and the second the imaginary part.

The fifth type is logical constants. These can only have one of two values:

```
1       .TRUE.
2       .FALSE.
```

Note that the dots enclosing the letters are required.

The last type is character constants. These are most often used as an *array* of characters, called a *string*. These consist of an arbitrary sequence of characters enclosed in apostrophes (single quotes):

```
1       'ABC'
2       'Anything goes!'
3       'It is a nice day'
```

Strings and character constants are case sensitive. A problem arises if you want to have an apostrophe in the string itself. In this case, you should double the apostrophe:

```
1       'It''s a nice day'
```

6.2 Expressions

The simplest non-constant expressions are of the form

```
1       operand operator operand
```

and an example is

```
1       x + y
```

The result of an expression is itself an operand, hence we can nest expressions together like

```
1       x + 2 * y
```

This raises the question of precedence: Does the last expression mean `x + (2*y)` or `(x+2)*y`? The precedence of arithmetic operators in Fortran 77 are (from highest to lowest):

```
1       **   {exponentiation}
2       *,/  {multiplication, division}
3       +,-  {addition, subtraction}
```

Operators of the same precedence are evaluated left-to-right, except the exponentiation operator **, which is evaluated right-to-left: `2**3**2` means `2**(3**2)` = 512. If you want to change the default evaluation order, you can use parentheses.

The above operators are all binary operators. There is also the unary operator `-` for negation. It has the *same* precedence as binary `+` and `-`,

which means it is applied *after* exponentiation, multiplication and division. An expression like `-x+y` means what you would expect, but beware of `-2**2`: it is `-(2**2)` $= -4$, not 4. Write `(-2)**2` if you mean the square of -2. Also note that two operators may not follow each other directly: `x*-y` is not standard Fortran 77, write `x*(-y)` instead.

Extreme caution must be taken when using the division operator, which has a quite different meaning for integers and reals. If the operands are both integers, an integer division is performed, otherwise a real arithmetic division is performed. For example, 3/2 equals 1, while 3./2. equals 1.5 (note the decimal points).

6.3 Assignment

The assignment has the form

```
1       variable_name = expression
```

The interpretation is as follows: Evaluate the right hand side and assign the resulting value to the variable on the left. The expression on the right may contain other variables, but these never change value! For example,

```
1       area = pi * r**2
```

does not change the value of `pi` or `r`, only `area`.

6.4 Type conversion

When different data types occur in the same expression, *type conversion* has to take place, either explicitly or implicitly. Fortran will do some type conversion implicitly. For example,

```
1       real x
2       x = x + 1
```

will convert the integer one to the real number one, and has the desired effect of incrementing x by one. However, in more complicated expressions, it is good programming practice to force the necessary type conversions explicitly. For numbers, the following functions are available:

```
1       int
2       real
3       dble
4       ichar
5       char
```

The first three have the obvious meaning. `ichar` takes a character and converts it to an integer, while `char` does exactly the opposite.

Example: How to multiply two real variables `x` and `y` using double precision and store the result in the double precision variable `w`:

```
1       w = dble(x)*dble(y)
```

Note that this is different from

```
1       w = dble(x*y)
```

In the first case both numbers are converted to double precision and the product is computed in double precision. In the second case the product is computed in single precision (and rounded to about 7 digits) and only then converted; the digits lost in the multiplication do not come back. The following program shows the difference:

```
1       program dblmul
2       implicit none
3       real x, y
4       double precision w1, w2
5
6       x = 4097.0
7       y = 4097.0
8       w1 = dble(x)*dble(y)
9       w2 = dble(x*y)
10      write (*,*) 'dble(x)*dble(y) =', w1
11      write (*,*) 'dble(x*y)       =', w2
12
13      x = 1.1
14      write (*,*) 'dble(1.1)       =', dble(x)
15      write (*,*) '1.1D0           =', 1.1D0
16
17      stop
18      end
```

Output with gfortran:

```
dble(x)*dble(y) =   16785409.000000000
dble(x*y)       =   16785408.000000000
dble(1.1)       =   1.1000000238418579
1.1D0           =   1.1000000000000001
```

The exact product $4097^2 = 16\,785\,409$ needs 25 binary digits, but a `real` only has 24, so `x*y` is rounded to 16 785 408. The last two lines show an even more common trap: converting a `real` to double precision does not make it more accurate. The real number 1.1 was already rounded when it was stored, and `dble` keeps that error. If you need a double precision constant, write it with a `D` exponent (`1.1D0`) from the start.

(If you use an old 32-bit compiler, you may see the same value on the first two lines: such compilers often compute intermediate results with extra precision. With gfortran you can avoid this with the options `-msse2 -mfpmath=sse`.)

6.5 Character strings and string manipulation

In Fortran 77, character strings are a distinct data type. Character variables can store a single character or a string of characters. While we briefly mentioned character constants earlier, here we will explore character variables and string manipulation in more detail.

6.5.1 Character declaration

Character variables must be explicitly declared with a length specification:

```
1       character*n variable_name
```

where n is the number of characters to be stored. For example:

```
1       character*10 name
2       character*80 line
3       character*1 answer
```

The character declaration may apply to a list of variables, all having the same length:

```
1       character*20 fname, lname
```

Alternatively, individual length can be specified for each variable:

```
1       character fname*20, lname*30, initial*1
```

6.5.2 Character arrays

Arrays of character strings can be declared in a similar manner to numeric arrays:

```
1       character*80 text(100)
```

This declares an array of 100 strings, each of length 80 characters.

6.5.3 Character assignment

Character assignment works like any other assignment in Fortran:

```
1       character*10 name
2       name = 'John Smith'
```

If the length of the string on the right is shorter than the declared length of the character variable, the value is padded with blanks on the right. If the length of the string on the right is longer than the declared length, the string is truncated on the right.

```
1       character*10 name1, name2
2       name1 = 'John'         ! 'John      ' (padded with 6 blanks)
3       name2 = 'John Smith'   ! 'John Smith'
4       name1 = 'Alexander Graham Bell'  ! 'Alexander ' (truncated)
```

6.5.4 String operations

Fortran 77 offers a few basic operations for string manipulation:

Concatenation

The concatenation operator is the double slash (//):

```
      character*5 first
      character*7 last
      character*13 fullname

      first = 'John'
      last = 'Smith'
      fullname = first // ' ' // last   ! Result: 'John  Smith  '
```

Notice the *two* blanks between the names: `first` has length 5, so it contains `'John '` with a trailing blank, and all 5 characters take part in the concatenation. This is one of the most common surprises with Fortran strings. To get `'John Smith'` you must use only the significant part of `first`, e.g. `first(1:4) // ' ' // last`. In Section 6.5.7 we show how to find the length of the significant part of a string.

String comparison

Character strings can be compared using the standard relational operators: .LT., .LE., .EQ., .NE., .GT., and .GE. When two strings of different length are compared, the shorter one is padded with blanks.

```
      character*5 word1, word2
      logical result

      word1 = 'alpha'
      word2 = 'beta'
      result = word1 .LT. word2    ! result = .TRUE.
```

The order used by these operators is the *collating sequence* of the computer. Fortran 77 only guarantees that letters and digits are in their natural order; whether, for example, `'a'` comes before or after `'B'` or `'1'` depends on the machine. Nowadays practically all computers use ASCII, but if you need a portable comparison in ASCII order, use the intrinsic functions `LLT`, `LLE`, `LGT` and `LGE` ("lexically less than", etc.):

```
      result = LLT(word1, word2)   ! same as word1 .LT. word2 in ASCII
```

6.5.5 String intrinsic functions

Fortran 77 provides several intrinsic functions for string manipulation:

LEN

Returns the declared length of a character variable:

```
      character*10 name
      integer length

      name = 'John'
      length = LEN(name)   ! length = 10, not 4
```

INDEX

Searches for a substring within a string and returns its position:

```
      character*20 text
      integer pos

      text = 'This is a test'
      pos = INDEX(text, 'is')    ! pos = 3 (first occurrence)
      pos = INDEX(text, 'xyz')   ! pos = 0 (not found)
```

Note that the first occurrence of `'is'` is inside the word `'This'`. (Fortran 90 added an optional third argument, `INDEX(text, 'is', .TRUE.)`, which searches from the end and would return 6; it is not available in Fortran 77.)

CHAR and ICHAR

These functions convert between characters and their integer codes. On practically all computers the codes are the ASCII values, but the standard does not guarantee it (`IACHAR` and `ACHAR`, which always use ASCII, only appeared in Fortran 90):

```
      character ch
      integer code

      ch = CHAR(65)      ! ch = 'A'
      code = ICHAR('A')  ! code = 65
```

6.5.6 Substring operations

In Fortran 77, you can extract a portion of a string using substring notation:

```
      character*10 name
      character*5 first

      name = 'John Smith'
      first = name(1:4)     ! first = 'John'
```

The general form of substring notation is:

```
      string(start:end)
```

Both *start* and *end* must be integers with $1 \leq start \leq end \leq$ the length of the string. If *start* is omitted, 1 is assumed. If *end* is omitted, the length of the string is assumed. The length of the substring is $end - start + 1$.

```
      character*20 text

      text = 'abcdefghijklmnopqrst'
      write(*,*) text(:5)     ! 'abcde'
      write(*,*) text(6:10)   ! 'fghij'
      write(*,*) text(16:)    ! 'pqrst'
```

Substring can also appear on the left side of an assignment:

```
      character*15 name

      name = 'John Doe'
      name(6:10) = 'Smith'   ! name becomes 'John Smith'
```

Be careful to give the substring the right length: `name(6:8)` has only 3 characters, so `name(6:8) = 'Smith'` would store just `'Smi'`, giving `'John Smi'`. As in any character assignment, the value is truncated (or padded with blanks) to fit.

6.5.7 Example: String processing

Here is a simple example that converts a string to uppercase:

```
      program strup
      implicit none
      character*80 text
      character*26 lower, upper
      integer i, k, n
      parameter (lower = 'abcdefghijklmnopqrstuvwxyz',
     +           upper = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ')

c Read input text
      write(*,*) 'Enter a string:'
      read(*,'(A)') text

c Find the actual length n of the text (ignoring trailing blanks)
      n = 0
      do 10 i = 80, 1, -1
         if (text(i:i) .NE. ' ') then
            n = i
            goto 20
         endif
   10 continue
   20 continue

c Convert lowercase to uppercase: look up each character in
c the string lower; if found, take the letter at the same
c position in upper.
      do 30 i = 1, n
```

```
27          k = INDEX(lower, text(i:i))
28          if (k .GT. 0) text(i:i) = upper(k:k)
29    30 continue
30
31  c Print the result
32        write(*,*) 'Uppercase version:'
33        if (n .GT. 0) write(*,'(A)') text(1:n)
34
35        stop
36        end
```

Running the program:

```
 Enter a string:
Hello, World!
 Uppercase version:
HELLO, WORLD!
```

This program reads a line of text, determines its actual length (ignoring trailing blanks), and converts any lowercase letters to uppercase. Some remarks:

- A common way to convert a letter is `CHAR(ICHAR(c) - 32)`, which works because in ASCII each lowercase letter comes exactly 32 positions after its uppercase version. The method used here, looking the letter up with `INDEX`, does not depend on the character codes at all, so it works on any computer.
- The variable holding the length is called `n`, not `len`. Using `len` would be legal (Fortran has no reserved words), but it would hide the intrinsic function `LEN` in this program.
- If the input line is empty, `n` stays 0. A substring `text(1:0)` has length zero, which Fortran 77 does not allow, so we only print when `n` is positive.

6.6 Exercises

Solutions are in the directory `code/exercises/ch06` that accompanies this tutorial.

1. Write a program that reads a number of seconds and prints it as hours, minutes and seconds (for example, 3725 seconds = 1 h 2 min 5 s), using only integer arithmetic.
2. Write a program that reads a line with a first name and a last name separated by one blank, for example `Grace Hopper`, and prints `Hopper, Grace`. Use `INDEX` to find the blank, and substrings to extract the names.

3. Predict the value of each of these expressions, and then check your predictions with a program: `2**3**2`, `-3**2`, `7/2*2`, `7/2.0*2`, `2+3*4**2/8` and `(2+3)*4**2/8`.

4. The volume of a sphere is $\frac{4}{3}\pi r^3$. Write a program that reads r and computes the volume twice, once with the expression `4/3*pi*r**3` and once with `4.0/3.0*pi*r**3`. Why are the results different? Which one is correct?

5. Write a program that computes 1/3 as a `real`, converts it to double precision with `dble`, and compares the result with 1/3 computed directly in double precision (`1.0D0/3.0D0`). Explain the difference.

6. Write a program that reads a date in the form `YYYY-MM-DD` (for example `2026-10-04`) into a `character*10` variable, and prints it in the form `DD/MM/YYYY`, using substrings and concatenation. Read the line with `read (*,'(A)')`.

7. Predict what this program prints, and then check:

```
      program concat
      implicit none
      character*6 a, b
      character*12 c, d

      a = 'Hi'
      b = 'there'
      c = a // b
      d = a(1:2) // ' ' // b(1:5)

      write (*,*) '[', c, ']'
      write (*,*) '[', d, ']'
      write (*,*) 'len(c) =', len(c), '   len(a // b) =', len(a //
     b)

      stop
      end
```

8. Write a program that reads an e-mail address, such as `grace.hopper@navy.mil`, and prints separately the user name (the part before the `@`) and the domain (the part after it). Use `INDEX` to find the `@` and the first blank after the address.

Chapter 7

Logical expressions

Logical expressions can only have the value `.TRUE.` or `.FALSE.`. A logical expression can be formed by comparing arithmetic expressions using the following *relational operators*:

```
      .LT.  meaning <
      .LE.  meaning <=
      .GT.  meaning >
      .GE.  meaning >=
      .EQ.  meaning =
      .NE.  meaning /=
```

So you *cannot* use symbols like $<$ or $>$ for comparison in Fortran 77, but you have to use the correct two-letter abbreviation enclosed by dots!

Logical expressions can be combined by the *logical operators* `.AND.` `.OR.` `.NOT.` which have the obvious meaning.

7.1 Logical variables and assignment

Truth values can be stored in *logical variables*. The assignment is analogous to the arithmetic assignment. Example:

```
      logical a, b
      a = .TRUE.
      b = a .AND. 3 .LT. 5/2
```

The order of precedence is important, as the last example shows. The rule is that arithmetic expressions are evaluated first, then relational operators, and finally logical operators. Hence b will be assigned `.FALSE.` in the example above. Among the logical operators the precedence (in the absence of parenthesis) is that `.NOT.` is done first, then `.AND.`, then `.OR.` is done last.

Logical variables are seldom used in Fortran. But logical expressions are frequently used in conditional statements like the `if` statement.

Chapter 8

The `if` statements

An important part of any programming language are the *conditional statements*. The most common such statement in Fortran is the `if` statement, which actually has several forms. The simplest one is the logical `if` statement:

```
1       if (logical expression) executable statement
```

This has to be written on one line. This example finds the absolute value of x:

```
1       if (x .LT. 0) x = -x
```

If more than one statement should be executed inside the `if`, then the following syntax should be used:

```
1       if (logical expression) then
2          statements
3       endif
```

The most general form of the `if` statement has the following form:

```
1       if (logical expression) then
2          statements
3       elseif (logical expression) then
4          statements
5        :
6        :
7       else
8          statements
9       endif
```

The execution flow is from top to bottom. The conditional expressions are evaluated in sequence until one is found to be true. Then the associated statements are executed and the control resumes after the `endif`.

8.1 Nested `if` statements

`if` statements can be nested in several levels. To ensure readability, it is important to use proper indentation. Here is an example:

```
      if (x .GT. 0) then
         if (x .GE. y) then
            write(*,*) 'x is positive and x >= y'
         else
            write(*,*) 'x is positive but x < y'
         endif
      elseif (x .LT. 0) then
         write(*,*) 'x is negative'
      else
         write(*,*) 'x is zero'
      endif
```

You should avoid nesting many levels of `if` statements since things get hard to follow.

8.2 Exercises

Solutions are in the directory `code/exercises/ch08` that accompanies this tutorial.

1. Write a program that reads three numbers and prints the largest one, using only `if` statements.

2. Write a program that finds the real roots of the equation $ax^2+bx+c=0$. Your program should distinguish four cases: two different roots, one (double) root, no real roots, and $a=0$ (not a quadratic equation). The square root is computed by the intrinsic function `sqrt`.

3. A year is a leap year if it is divisible by 4, except the years divisible by 100, which are leap years only if they are also divisible by 400. Write a program that reads a year and says whether it is a leap year, using a single logical expression stored in a `logical` variable. (The remainder of the division of `a` by `b` is `mod(a, b)`.) Test it with 1900, 2000, 2024 and 2026.

4. Write a program that reads a score between 0 and 100 and prints the corresponding letter grade: A for 90 or more, B for 80 to 89, C for 70 to 79, D for 60 to 69 and F below 60. Scores outside the range 0–100 should be rejected with a message. Use an `if-elseif` chain.

Chapter 9

Loops

For repeated execution of similar things, *loops* are used. If you are familiar with other programming languages you have probably heard about *for*-loops, *while*-loops, and *until*-loops. Fortran 77 has only one loop construct, called the `do`-loop. The `do`-loop corresponds to what is known as a *for*-loop in other languages. Other loop constructs have to be built using the `if` and `goto` statements.

9.1 do-loops

The `do`-loop is used for simple counting. Here is a simple example that prints the cumulative sums of the integers from 1 through n (assume n has been assigned a value elsewhere):

```
1       integer i, n, sum
2
3       sum = 0
4       do 10 i = 1, n
5          sum = sum + i
6          write(*,*) 'i =', i
7          write(*,*) 'sum =', sum
8    10 continue
```

The number 10 is a statement *label*. Typically, there will be many loops and other statements in a single program that require a statement label. The programmer is responsible for assigning a unique number to each label in each program (or subprogram). Recall that column positions 1-5 are reserved for statement labels. The numerical value of statement labels have no significance, so any integers can be used, in any order. Typically, most programmers use consecutive multiples of 10.

The variable defined in the `do`-statement is incremented by 1 by default. However, you can define the *step* to be any number but zero. This program segment prints the even numbers between 1 and 10 in decreasing order:

```
1       integer i
2
3       do 20 i = 10, 1, -2
4          write(*,*) 'i =', i
5    20 continue
```

The general form of the `do` loop is as follows:

```
1       do label  var =  expr1, expr2, expr3
2          statements
3  label continue
```

var is the loop variable (often called the *loop index*) which must be integer. *expr1* specifies the initial value of *var*, *expr2* is the terminating bound, and *expr3* is the increment (step).

Note: The `do`-loop variable must never be changed by other statements within the loop! The standard forbids it, and the result would be very confusing.

Fortran 77 also allows the loop index to be of type `real`, but due to round off errors it may not take on exactly the expected sequence of values. Real loop indices were removed from the language in Fortran 95: always use integers.

9.1.1 How many times is the loop executed?

Before the first pass through the loop, Fortran computes the *iteration count*

$$\max\left(0,\ \left\lfloor \frac{expr2 - expr1 + expr3}{expr3} \right\rfloor\right)$$

and then executes the body exactly that many times. Two consequences are worth remembering:

- If the count is zero, the body is not executed at all. For example, `do 10 i = 1, n` does nothing when `n` is 0. (In the older Fortran 66 standard, many compilers executed such a loop once. If you maintain very old code, keep this in mind.)
- After the loop is finished, the loop variable has the value it would have had in the next iteration, i.e. one step beyond the last value used.

The following program illustrates both points:

```
1       program trips
2       implicit none
3       integer i, n
4
5       n = 0
6       do 10 i = 1, n
7          write (*,*) 'This line is never printed'
```

```
8     10 continue
9        write (*,*) 'After the loop, i =', i
10
11       do 20 i = 10, 1, -3
12          write (*,*) 'i =', i
13    20 continue
14       write (*,*) 'After the loop, i =', i
15
16       stop
17       end
```

Output:

```
After the loop, i =           1
i =          10
i =           7
i =           4
i =           1
After the loop, i =          -2
```

In the second loop the count is $\lfloor (1 - 10 - 3)/(-3) \rfloor = 4$.

Many Fortran 77 compilers allow `do`-loops to be closed by the `enddo` statement. The advantage of this is that the statement label can then be omitted since it is assumed that an `enddo` closes the nearest previous `do` statement. The `enddo` construct is widely used, but it is not a part of ANSI Fortran 77.

This also explains why, unlike in some programming languages, changing the variables used in the start, end, and step expressions inside the loop has no effect on the number of iterations: they are evaluated only once, before the first pass through the body of the loop. This means that the following do-loop will multiply a non-negative j by two (the hard way), rather than running forever as the equivalent loop might in another language.

```
1        integer i,j
2
3        read (*,*) j
4        do 20 i = 1, j
5           j = j + 1
6     20 continue
7        write (*,*) j
```

9.2 while-loops

The most intuitive way to write a `while`-loop is

```
1        do while (logical expr)
2           statements
3        enddo
```

The program will alternate testing the condition and executing the statements in the body as long as the condition in the `while` statement is true.

Even though this syntax is accepted by most compilers (and is standard since Fortran 90), it is not ANSI Fortran 77. The correct way is to use `if` and `goto`:

```
label if (logical expr) then
         statements
         goto label
      endif
```

Here is an example that calculates and prints all the powers of two that are less than or equal to 100:

```
      integer n

      n = 1
   10 if (n .le. 100) then
         write (*,*) n
         n = 2*n
         goto 10
      endif
```

Remember the column rules: the label `10` goes in columns 1–5, and the statement itself must start in column 7 or later. A statement that starts in column 6 would be taken as a continuation line, and the compiler would complain.

9.3 until-loops

If the termination criterion is at the end instead of the beginning, it is often called an until-loop. The pseudocode looks like this:

```
      do
         statements
      until (logical expr)
```

Again, this should be implemented in Fortran 77 by using `if` and `goto`:

```
label continue
         statements
      if (logical expr) goto label
```

Note that the logical expression in the latter version should be the negation of the expression given in the pseudocode!

9.4 Advanced control statements

In addition to the basic `if` statements and `do` loops that we have already covered, Fortran 77 provides several other control structures. These include arithmetic `if`, computed `goto`, assigned `goto`, and other variations. While these constructs are less commonly used in modern programming style (and

are often discouraged), they are part of the Fortran 77 standard and you may encounter them in older code.

9.4.1 Arithmetic `if` statement

The arithmetic `if` statement is a three-way branch based on whether an arithmetic expression is negative, zero, or positive:

```
      if (expr) label1, label2, label3
```

Control will transfer to *label1* if $expr < 0$, to *label2* if $expr = 0$, and to *label3* if $expr > 0$.

For example:

```
      if (x - y) 10, 20, 30
 10   write(*,*) 'x is less than y'
      goto 40
 20   write(*,*) 'x equals y'
      goto 40
 30   write(*,*) 'x is greater than y'
 40   continue
```

This is an older form of `if` statement and is generally considered less readable than the logical `if` or block `if` statements. It should be avoided in new code. It was declared obsolescent in Fortran 90 and deleted from the language in Fortran 2018; gfortran still accepts it, with a warning.

9.4.2 Unconditional `goto`

We have already seen the unconditional `goto` in earlier examples. It transfers control to the statement with the specified label:

```
      goto label
```

For example:

```
      goto 100
      ...
 100  continue
```

While `goto` statements are sometimes necessary in Fortran 77 (especially for implementing certain loop structures), they should be used sparingly as they can make programs difficult to understand and maintain.

9.4.3 Computed `goto`

The computed `goto` statement provides a way to branch to one of several labels based on the value of an integer expression:

```
      goto (label1, label2, ..., labelN) expr
```

If *expr* = 1, control transfers to *label1*; if *expr* = 2, control transfers to *label2*; and so on. If *expr* is less than 1 or greater than *N*, the computed `goto` will not transfer control, and execution will continue with the next statement.

Here's an example. To see what happens when `day` is out of range, the program tries all values from 0 to 8:

```
1       program weekdy
2       implicit none
3       integer day
4
5       do 90 day = 0, 8
6          write(*,*) 'day =', day
7          goto (10, 20, 30, 40, 50, 60, 70) day
8          write(*,*) '   (out of range: no jump)'
9          goto 80
10    10   write(*,*) '   Sunday'
11         goto 80
12    20   write(*,*) '   Monday'
13         goto 80
14    30   write(*,*) '   Tuesday'
15         goto 80
16    40   write(*,*) '   Wednesday'
17         goto 80
18    50   write(*,*) '   Thursday'
19         goto 80
20    60   write(*,*) '   Friday'
21         goto 80
22    70   write(*,*) '   Saturday'
23    80   continue
24    90 continue
25
26       stop
27       end
```

Output (shortened):

```
day =           0
   (out of range: no jump)
day =           1
   Sunday
day =           2
   Monday
  ...
day =           7
   Saturday
day =           8
   (out of range: no jump)
```

Notice that every branch must end with `goto 80`; otherwise execution would simply "fall through" into the next branch.

In modern code, this would typically be implemented using a block `if` statement or, in Fortran 90 and later, a `select case` statement.

9.4.4 Assigned `goto`

The assigned `goto` statement is a more complex variation where a label is first assigned to an integer variable using the `assign` statement, and then control is transferred to that label:

```
      assign label to int_var
      ...
      goto int_var [,[,] (label1, label2, ..., labelN)]
```

The optional list of labels in parentheses specifies the valid target labels, but this is not required in standard Fortran 77.

Here's an example:

```
      integer jump
      ...
      assign 100 to jump
      ...
      goto jump
      ...
  100 continue
```

Or with the optional label list:

```
      integer route
      ...
      assign 200 to route
      ...
      goto route, (100, 200, 300)
      ...
  100 continue
      ...
  200 continue
      ...
  300 continue
```

This construct is rarely used and should be avoided in new code. The `assign` statement and the assigned `goto` were deleted from the language in Fortran 95; gfortran still accepts them, but with a warning.

9.4.5 Alternate `return`

Fortran 77 allows a subroutine to return to different points in the calling program using the alternate `return` feature. This is specified in the subroutine's parameter list and in the `return` statement:

```
c In the subroutine: each * marks an alternate return point
      subroutine sub(arg1, arg2, *, *)
      ...
      return expr
      ...
      end

```

```
c In the caller: the actual arguments are *label
      call sub(x, y, *label1, *label2)
```

If *expr* = 1, control returns to *label1* in the calling program; if *expr* = 2, control returns to *label2*; and so on. A plain `return` without an expression returns to the statement following the subroutine call.

Here's an example:

```
      program altret
      implicit none
      integer k

      do 400 k = 1, 3
         call check(k, *100, *200)
         write(*,*) k, ': normal return'
         goto 300
  100    write(*,*) k, ': error'
         goto 300
  200    write(*,*) k, ': end of data'
  300    continue
  400 continue

      stop
      end

      subroutine check(code, *, *)
      implicit none
      integer code

      if (code .eq. 2) return 1
      if (code .eq. 3) return 2
      return
      end
```

Output:

```
           1 : normal return
           2 : error
           3 : end of data
```

This feature should also be avoided in modern code, as it can make program flow difficult to follow.

9.4.6 Non-block `do`

Although we've covered the standard `do` loop construction in a previous lesson, it's worth noting that Fortran 77 allows the target statement of a `do` loop to be any executable statement, not just `continue`:

```
      do 50 i = 1, 10
      do 50 j = 1, 10
         a(i,j) = 0.0
  50  b(i,j) = 1.0
```

In this example, both loops share the same terminal statement (`b(i,j) = 1.0`), which belongs to the inner loop. This style is confusing and error-prone, and should be avoided: both shared termination and terminal statements other than `continue` (or `enddo`) were declared obsolescent in Fortran 90 and deleted in Fortran 2018. gfortran still accepts them, with a warning.

9.4.7 Best practices

While the advanced control statements described here are part of standard Fortran 77, they are generally considered to be less readable and maintainable than the more structured control statements (`if-then-else` and standard `do` loops). Here are some recommendations:

- Use block `if` statements instead of arithmetic `if`
- Use block `if` or `select case` (in Fortran 90+) instead of computed `goto`
- Avoid assigned `goto` entirely
- Use error codes or logical flags instead of alternate returns
- Always use `continue` as the target of `do` loops for clarity
- Avoid non-block `do` loops with shared terminal statements

By following these guidelines, your Fortran code will be more readable, maintainable, and less prone to errors.

9.5 Exercises

Solutions are in the directory `code/exercises/ch09` that accompanies this tutorial.

1. Write a program that prints $n!$ (the factorial of n) for $n = 1, 2, \ldots, 15$, computed both with an `integer` variable and with a `double precision` variable. From which n on are the integer results wrong, and why?
2. The greatest common divisor of two positive integers can be computed with Euclid's algorithm: $\gcd(a, 0) = a$, and $\gcd(a, b) = \gcd(b, a \bmod b)$. Write a program that reads a and b and computes their greatest common divisor, using a while-loop built with `if` and `goto`. The remainder $a \bmod b$ is given by the intrinsic function `mod(a, b)`. Check that $\gcd(1071, 462) = 21$.
3. Write a program that prints a table with the numbers from 1 to 10, their squares and their cubes.

4. The sum $\sum_{k=1}^{n} 1/k^2$ approaches $\pi^2/6$ as n grows. Write a program that computes the sum for $n = 1000$ in double precision and compares it with $\pi^2/6$. Be careful with the expression for the terms: `1/k**2` would be an integer division.

5. Without running them, how many times is the body of each of these loops executed: `do i = 1, 10, 3`; `do i = 10, 1`; `do i = 10, 1, -3`; `do i = 1, 1`? Check your answers with the formula for the iteration count, and then with a program that counts the iterations.

6. The Collatz sequence starts from a positive integer n; if n is even it is halved, otherwise it is replaced by $3n + 1$, and the process is repeated until n is 1. Write a program that reads n, prints the sequence, and counts the steps, using a while-loop built with `if` and `goto`. (Starting from 6 it takes 8 steps.)

7. Write a program that reads numbers and adds them until a negative number is read (the negative number is not added), and then prints how many numbers were added and their sum. Use an until-loop.

8. Rewrite the program `weekdy` of this chapter with a block `if` instead of the computed `goto`.

Chapter 10

Arrays

Many scientific computations use vectors and matrices. The data type Fortran uses for representing such objects is the *array*. A one-dimensional array corresponds to a vector, while a two-dimensional array corresponds to a matrix. To fully understand how this works in Fortran 77, you will have to know not only the syntax for usage, but also how these objects are stored in memory in Fortran 77.

10.1 One-dimensional arrays

The simplest array is the one-dimensional array, which is just a sequence of elements stored consecutively in memory. For example, the declaration

```
1       real a(20)
```

declares `a` as a real array of length 20. That is, `a` consists of 20 real numbers stored contiguously in memory. By convention, Fortran arrays are indexed from 1 and up. Thus the first number in the array is denoted by `a(1)` and the last by `a(20)`. However, you may define an arbitrary index range for your arrays using the following syntax:

```
1       real b(0:19), weird(-162:237)
```

Here, `b` is exactly similar to `a` from the previous example, except the index runs from 0 through 19. `weird` is an array of length 237-(-162)+1 = 400.

The type of an array element can be any of the basic data types. Examples:

```
1       integer i(10)
2       logical aa(0:1)
3       double precision x(100)
```

Each element of an array can be thought of as a separate variable. You reference the i'th element of array `a` by `a(i)`. Here is a code segment that stores the 10 first square numbers in the array `sq`:

```
      integer i, sq(10)

      do 100 i = 1, 10
         sq(i) = i**2
  100 continue
```

A common bug in Fortran is that the program tries to access array elements that are out of bounds or undefined. This is the responsibility of the programmer: the Fortran 77 standard does not require the compiler to detect such bugs, and by default the program just reads or overwrites whatever happens to be in memory next to the array. Most compilers can check the indices at run time, at some cost in speed; with gfortran, compile with `-fcheck=bounds` (included in `-fcheck=all`) while you develop and test your program (see the chapter "Debugging hints" and exercise 5 below).

10.2 Two-dimensional arrays

Matrices are very important in linear algebra. Matrices are usually represented by two-dimensional arrays. For example, the declaration

```
      real A(3,5)
```

defines a two-dimensional array of 3*5=15 real numbers. It is useful to think of the first index as the row index, and the second as the column index. Hence we get the graphical picture:

```
  (1,1)  (1,2)  (1,3)  (1,4)  (1,5)
  (2,1)  (2,2)  (2,3)  (2,4)  (2,5)
  (3,1)  (3,2)  (3,3)  (3,4)  (3,5)
```

Two-dimensional arrays may also have indices in an arbitrary defined range. The general syntax for declarations is:

```
      name (low_index1 : hi_index1, low_index2 : hi_index2)
```

The total size of the array is then

```
      size = (hi_index1-low_index1+1)*(hi_index2-low_index2+1)
```

It is quite common in Fortran to declare arrays that are larger than the matrix we want to store. (This is because Fortran does not have dynamic storage allocation.) This is perfectly legal. Example:

```
      real A(3,5)
      integer i,j
c
c     We will only use the upper 3 by 3 part of this array.
c
      do 20 j = 1, 3
         do 10 i = 1, 3
            a(i,j) = real(i)/real(j)
```

```
9     10    continue
10    20 continue
```

The elements in the submatrix A(1:3,4:5) are undefined. Do not assume these elements are initialized to zero by the compiler (some compilers will do this, but not all).

10.3 Storage format for 2-dimensional arrays

Fortran stores higher dimensional arrays as a contiguous sequence of elements. It is important to know that 2-dimensional arrays are stored *by column*. So in the above example, array element (1,2) will follow element (3,1). Then follows the rest of the second column, thereafter the third column, and so on.

Consider again the example where we only use the upper 3 by 3 submatrix of the 3 by 5 array `A(3,5)`. The 9 interesting elements will then be stored in the first nine memory locations, while the last six are not used. This works out neatly because the *leading dimension* is the same for both the array and the matrix we store in the array. However, frequently the leading dimension of the array will be larger than the first dimension of the matrix. Then the matrix will *not* be stored contiguously in memory, even if the array is contiguous. For example, suppose the declaration was `A(5,3)` instead. Then there would be two "unused" memory cells between the end of one column and the beginning of the next column (again we are assuming the matrix is 3 by 3).

This may seem complicated, but actually it is quite simple when you get used to it. If you are in doubt, it can be useful to look at how the *address* of an array element is computed. Each array will have some memory address assigned to the beginning of the array, that is element (1,1). The address of element (i,j) is then given by

```
1       addr[A(i,j)] = addr[A(1,1)] + (j-1)*lda + (i-1)
```

where `lda` is the leading (i.e. row) dimension of `A`. Note that `lda` is in general different from the actual matrix dimension. Many Fortran errors are caused by this, so it is very important you understand the distinction!

10.4 Multi-dimensional arrays

Fortran 77 allows arrays of up to seven dimensions. The syntax and storage format are analogous to the two-dimensional case, so we will not spend time on this.

10.5 The `dimension` statement

There is an alternate way to declare arrays in Fortran 77. The statements

```
1       real A, x
2       dimension x(50)
3       dimension A(10,20)
```

are equivalent to

```
1       real A(10,20), x(50)
```

This `dimension` statement is considered old-fashioned style today.

10.6 Exercises

Solutions are in the directory `code/exercises/ch10` that accompanies this tutorial.

1. Write a program that reads an integer $n \leq 100$ and then n real numbers into an array, and prints their mean and all the numbers that are larger than the mean. Why do you need an array for this, while the mean alone could be computed without one?

2. Write a program that fills a 3×4 integer matrix with $A(i, j) = 10i + j$, stores its transpose in a 4×3 matrix B, and prints both matrices row by row.

3. An array is declared as `real A(5,3)`, and only the 3×3 matrix in its upper part is used. Counting the memory positions of the array from 1, in which position is `A(2,3)` stored? Which positions are not used? (Answer without a computer.)

4. Write a program that reads an integer $n \leq 100$ and then n real numbers into an array, and prints the largest number and its position in the array.

5. Compile the program `checks/bounds.f` that accompanies this tutorial, first without options and then with `-fcheck=bounds`, and run it both times. What is the error, and why does the program seem to work in the first case? Fix it.

Chapter 11

Subprograms

When a program is more than a few hundred lines long, it gets hard to follow. Fortran codes that solve real engineering problems often have tens of thousands of lines. The only way to handle such big codes, is to use a *modular* approach and split the program into many separate smaller units called *subprograms*.

A subprogram is a (small) piece of code that solves a well defined subproblem. In a large program, one often has to solve the same subproblems with many different data. Instead of replicating code, these tasks should be solved by subprograms. The same subprogram can be invoked many times with different input data.

Fortran has two different types of subprograms, called *functions* and *subroutines*.

11.1 Statement functions

Before we discuss external functions and subroutines, let's look at a simpler construct called the *statement function*. A statement function is defined by a single statement within the program unit that uses it. It provides a convenient way to define simple, single-line functions.

The syntax for a statement function is:

```
1       name(dummy_arg_list) = expression
```

Here is our circle program once more, now with a statement function `carea` that calculates the area of a circle:

```
1       program circle
2       implicit none
3       real r, area, pi, rad, carea
4       parameter (pi = 3.14159)
5
6   c Statement function: area of a circle with radius rad
7       carea(rad) = pi * rad * rad
```

```
8
9  c Main program
10       write(*,*) 'Give radius r:'
11       read(*,*) r
12       area = carea(r)
13       write(*,*) 'Area = ', area
14
15       stop
16       end
```

The dummy argument `rad` is only a placeholder: when we write `carea(r)`, the expression `pi * rad * rad` is evaluated with `rad` replaced by `r`. Both the function name and the dummy argument must have a type, which is why they appear in the `real` declaration.

Statement functions have several restrictions:

- They must be defined after all specification statements (`parameter`, `dimension`, etc.) but before any executable statements
- The expression may use the dummy arguments, constants, and variables of the surrounding program unit
- The definition is a single expression, so it cannot contain statements such as `if` or `do`
- The expression may call intrinsic and external functions, and statement functions defined *earlier* in the same program unit
- They cannot be recursive, and they cannot be passed as arguments to other subprograms

Statement functions are useful for simple calculations that are needed repeatedly within a program unit, but for more complex operations, external functions or subroutines are necessary.

11.2 Functions

Fortran functions are quite similar to mathematical functions: They both take a set of input arguments (parameters) and return a value of some type. In the preceding discussion we talked about *user defined* subprograms. Fortran 77 also has some *intrinsic* (built-in) functions.

A simple example illustrates how to use a function:

```
1        x = cos(pi/3.0)
```

Here `cos` is the cosine function, so `x` will be assigned the value 0.5 (if `pi` has been correctly defined; Fortran 77 has no built-in constants). There are many intrinsic functions in Fortran 77. Some of the most common are:

```
      abs      absolute value
      min      minimum value
      max      maximum value
      sqrt     square root
      sin      sine
      cos      cosine
      tan      tangent
      atan     arctangent
      exp      exponential (natural)
      log      logarithm (natural)
```

In general, a function always has a *type*. Most of the built-in functions mentioned above, however, are *generic*. So in the example above, `pi` and `x` could be either of type `real` or `double precision`. The compiler would check the types and use the correct version of `cos` (real or double precision). Unfortunately, Fortran is not really a *polymorphic* language so in general you have to be careful to match the types of your variables and your functions!

Now we turn to the user-written functions. Consider the following problem: in a region with wet winters and dry summers, a hydrologist describes the rainfall of each month with a simple model *mrain(m,t)*, where m is the month (1 to 12) and t is a parameter that depends on the location. Given the formula for *mrain* and the value of t, compute the annual rainfall.

The obvious way to solve the problem is to write a loop that runs over all the months and sums up the monthly values. Since computing the rainfall of one month is an independent subproblem, it is convenient to implement it as a function. The following main program can be used:

```
      program annual
      implicit none
      real mrain, t, r, sum
      integer m

      read (*,*) t
      sum = 0.0
      do 10 m = 1, 12
         r = mrain(m, t)
         write (*,*) 'Month', m, ':', r
         sum = sum + r
   10 continue
      write (*,*) 'Annual rainfall is ', sum, ' inches'

      stop
      end
```

Note that we have declared `mrain` to be `real` just as we would a variable. In addition, `mrain` has to be defined as a Fortran function. The model is

$$mrain(m,t) = \begin{cases} \frac{t}{10}\left(m^2 - 13m + 30\right) & \text{if this is positive,} \\ 0 & \text{otherwise.} \end{cases}$$

(The polynomial is negative from April to September: those are the dry months.) The corresponding Fortran function is

```
      real function mrain(m, t)
      implicit none
      integer m
      real t

      mrain = 0.1*t*(m**2 - 13*m + 30)
      if (mrain .LT. 0.0) mrain = 0.0

      return
      end
```

With $t = 1.5$ the program prints

```
Month           1 :   2.70000005
Month           2 :   1.20000005
Month           3 :   0.00000000
  ...
Month          11 :   1.20000005
Month          12 :   2.70000005
Annual rainfall is    7.80000019      inches
```

We see that the structure of a function closely resembles that of the main program. The main differences are:

- Functions have a type. This type must also be declared in the calling program.
- The return value is stored in a variable with the same name as the function.
- Functions return to the calling program unit, instead of stopping the program. This happens when a `return` statement or the final `end` is reached; the `return` just before `end` is therefore optional, but it is customary.

To sum up, the general syntax of a Fortran 77 function is:

```
      type function name (list-of-variables)
      declarations
      statements
      return
      end
```

The function is called by simply using the function name and listing the arguments in parentheses.

11.2.1 Why the function must be declared

The function has to be declared with the correct type in the calling program unit. If you use a function which has not been declared, Fortran will use

the same implicit typing rules as for variables. Here the name starts with `m`, so `mrain` would be taken to be an `integer` function. Suppose we remove both `implicit none` and `mrain` from the declarations of the main program, and put the main program and the function in separate files, `annual.f` and `mrain.f`. The program compiles without any complaint:

```
gfortran annual.f mrain.f -o annual
```

but the results are nonsense. With gfortran on a 64-bit Windows or Linux computer, for example, we get:

```
 Month            1 :   18.0000000
 Month            2 :   8.00000000
 Month            3 :   0.00000000
 Month            4 :  -6.00000000
   ...
 Month           12 :   18.0000000
 Annual rainfall is   -4.00000000      inches
```

Negative rainfall! The function returns a real number, but the main program expects an integer and looks for it in the wrong place, where it finds whatever happens to be there (here, apparently, an intermediate result of the function). On other computers the numbers are different, for example huge values such as `4.28741760E+09`. The dangerous case is the one shown here, where the wrong results look almost plausible. The compiler cannot detect the error, because it compiles each file separately and never sees both the call and the function at the same time. (If both are in the same file, modern versions of gfortran do notice the mismatch.) With `implicit none` in the main program, however, the mistake is caught immediately:

```
Error: Function 'mrain' at (1) has no IMPLICIT type
```

This is one more good reason to always use `implicit none`.

It should be noted that strictly speaking Fortran 77 doesn't permit recursion (functions which call themselves). However, it is not uncommon for a compiler to allow recursion.

11.3 Subroutines

A Fortran function can essentially only return one value. Often we want to return two or more values (or sometimes none!). For this purpose we use the `subroutine` construct. The syntax is as follows:

```
1       subroutine name (list-of-arguments)
2       declarations
3       statements
4       return
5       end
```

Note that subroutines have no type and consequently should not (cannot) be declared in the calling program unit. They are also invoked differently than functions, using the word *call* before their names and parameters.

We give an example of a very simple subroutine. The purpose of the subroutine is to swap two integers.

```
1       subroutine iswap (a, b)
2       implicit none
3       integer a, b
4  c Local variables
5       integer tmp
6
7       tmp = a
8       a = b
9       b = tmp
10
11      return
12      end
```

Note that there are two blocks of variable declarations here. First, we declare the input/output parameters, i.e. the variables that are common to both the caller and the callee. Afterwards, we declare the *local variables*, i.e. the variables that can only be used within this subprogram. We can use the same variable names in different subprograms and the compiler will know that they are different variables that just happen to have the same names.

11.4 Call-by-reference

Fortran 77 uses the so-called *call-by-reference* paradigm. This means that instead of just passing the values of the function/subroutine arguments (*call-by-value*), the memory address of the arguments (pointers) are passed instead. A small example should show the difference:

```
1       program callex
2       implicit none
3       integer m, n
4  c
5       m = 1
6       n = 2
7
8       call iswap(m, n)
9       write(*,*) m, n
10
11      stop
12      end
```

The output from this program is "2 1", just as one would expect. However, if Fortran 77 had been using call-by-value then the output would have been "1 2", i.e. the variables m and n were unchanged! The reason for this is that only the values of m and n had been copied to the subroutine iswap,

and even if a and b were swapped inside the subroutine the new values would not have been passed back to the main program.

In the above example, call-by-reference was exactly what we wanted. But you have to be careful about this when writing Fortran code, because it is easy to introduce undesired *side effects*. For example, sometimes it is tempting to use an input parameter in a subprogram as a local variable and change its value. Since the new value will then propagate back to the calling program with an unexpected value, you should *never* do this unless (like our iswap subroutine) the change is part of the purpose of the subroutine.

11.4.1 A dangerous mistake: passing a constant

What happens if one of the arguments is a constant instead of a variable?

```
1       program swapk
2       implicit none
3       integer n
4
5       n = 5
6       call iswap(1, n)
7       write(*,*) 'n =', n
8
9       stop
10      end
```

The subroutine receives the *address* of the constant 1 and tries to store the value 5 there. The standard forbids this, but the compiler cannot detect it if `iswap` is in another file. What actually happens depends on the system. With gfortran on a current computer, constants are kept in read-only memory, and the program crashes:

```
Program received signal SIGSEGV: Segmentation fault -
invalid memory reference.
```

On many older systems the program did *not* crash: the constant was silently changed, so that from then on every `1` in the program had the value 5! Never pass a constant (or an expression) as an argument that the subprogram modifies.

We will come back to this issue in a later section on passing arrays as arguments (parameters).

11.5 The `SAVE` statement

In Fortran 77, the local variables of a subprogram become *undefined* when the subprogram returns: the standard does not guarantee that they still have their old values at the next call. (Many compilers do keep the values, which is why old programs that rely on it often seem to work, until they are compiled with another compiler or with different options.) However, there

are situations where you want a local variable to maintain its value between calls. This is where the `SAVE` statement comes in.

The syntax of the `SAVE` statement is:

```
1       SAVE [list-of-variables]
```

If no list is provided, all local variables in the subprogram are saved. Here's a simple example of using `SAVE` to create a counter that remembers how many times a subroutine has been called:

```
1       program calls
2       implicit none
3
4       call counter
5       call counter
6       call counter
7
8       stop
9       end
10
11      subroutine counter
12      implicit none
13      integer count
14      save count
15      data count /0/
16
17      count = count + 1
18      write(*,*) 'This subroutine has been called', count, 'times.'
19
20      return
21      end
```

Output:

```
This subroutine has been called           1 times.
This subroutine has been called           2 times.
This subroutine has been called           3 times.
```

The `DATA` statement gives `count` its initial value 0 once, when the program is loaded; it is not executed again at each call. Without the `SAVE` statement, the value of `count` would be undefined at the second and later calls.

Here's another example that uses `SAVE` to maintain state between function calls. The function `runsum` keeps a *running sum*: each call adds its argument to a total and returns the new total.

```
1       program total
2       implicit none
3       real runsum, s
4
5       s = runsum(10.0)
6       s = runsum(-2.5)
7       s = runsum(4.0)
8       write(*,*) 'Running sum =', s
```

```
9
10      stop
11      end
12
13      real function runsum(x)
14      implicit none
15      real x, sum
16      save sum
17      data sum /0.0/
18
19      sum = sum + x
20      runsum = sum
21
22      return
23      end
```

Output:

```
 Running sum =   11.5000000
```

The SAVE statement ensures that the variable sum maintains its value between function calls. Notice that runsum is not a "pure" function in the mathematical sense: runsum(4.0) returns a different value each time it is called.

11.5.1 When to use SAVE

You should use the SAVE statement when:

- A subprogram needs to remember state information between calls
- You have a variable initialized with a DATA statement that should not be reinitialized on subsequent calls
- You need to perform an expensive initialization once and reuse the results in subsequent calls

11.5.2 Restrictions on SAVE

There are some restrictions on the use of SAVE:

- Dummy arguments (parameters) cannot be saved.
- Individual variables of a common block cannot appear in a SAVE list; you can only save the whole block, by writing its name between slashes: save /coeff/. (A named common block keeps its values as long as some active program unit uses it; to be safe, save it in every subprogram that uses it, or declare it in the main program.)

- Variables initialized by a `DATA` statement are *not* automatically saved in Fortran 77: if they are changed, they become undefined on return unless they are saved. (In Fortran 90 initialized variables are saved automatically.)

11.5.3 Best practices with `SAVE`

Some recommendations for using `SAVE`:

- Always explicitly initialize saved variables, either with a DATA statement or within the executable code
- Document the use of SAVE variables clearly, as they create implicit state that can make debugging difficult
- Consider alternatives like passing state through parameters or using common blocks for more complex state
- Be aware that saved variables can make code non-reentrant and potentially not thread-safe

11.6 Exercises

Solutions are in the directory `code/exercises/ch11` that accompanies this tutorial.

1. Write a logical function `isprim(n)` which returns `.TRUE.` if the integer `n` is a prime number and `.FALSE.` otherwise. Use it in a program that prints all the prime numbers below 100 (there are 25 of them). Hint: it is enough to try the divisors k with $k^2 \leq n$.

2. Write a real function `rnd()` (a function may have no arguments) that returns a pseudo-random number between 0 and 1, using the *linear congruential* method: each call computes `seed = mod(25173*seed + 13849, 65536)` and returns `seed/65536.0`. The variable `seed` must keep its value between calls: use `SAVE`, and give it an initial value with `DATA`. Test the function by printing the average of 10000 numbers, which should be close to 0.5.

3. Write a program that prints a table of conversion from degrees Fahrenheit to degrees Celsius, for $0, 10, 20, \ldots, 100$ degrees Fahrenheit. Define the conversion $C = (F - 32) \cdot 5/9$ with a statement function.

4. Write a real function `dist(x1, y1, x2, y2)` that returns the distance between the points (x_1, y_1) and (x_2, y_2), and use it to compute the perimeter of the triangle with vertices $(0, 0)$, $(3, 0)$ and $(0, 4)$.

5. Write an integer function `ndigit(n)` that returns the number of decimal digits of an integer $n \geq 0$ (`ndigit(0)` is 1). The function must not change its argument: why?

6. Write a subroutine `polar(x, y, r, theta)` that converts the cartesian coordinates (x, y) of a point to polar coordinates $r = \sqrt{x^2 + y^2}$ and θ, in degrees. Use the intrinsic function `atan2(y, x)`, which returns the angle in radians, between $-\pi$ and π. Why is a subroutine needed here, and not a function?

7. Write a subroutine `sort3(a, b, c)` that sorts three integers in increasing order, using the subroutine `iswap` of this chapter.

8. What does the following program print? Why? Fix the function.

```
1       program sumto
2       implicit none
3       integer isum, n, s
4
5       n = 4
6       s = isum(n)
7       write (*,*) 'Sum of 1 to', n, ' is', s
8
9       stop
10      end
11
12      integer function isum(n)
13      implicit none
14      integer n
15
16      isum = 0
17    10 if (n .GT. 0) then
18         isum = isum + n
19         n = n - 1
20         goto 10
21      endif
22
23      return
24      end
```

Chapter 12

Arrays in subprograms

Fortran subprogram calls are based on *call by reference*. This means that the calling parameters are not copied to the called subprogram, but rather that the *addresses* of the parameters (variables) are passed. This saves a lot of memory space when dealing with arrays. No extra storage is needed as the subroutine operates on the same memory locations as the calling (sub-)program. However, you as a programmer have to know about this and take it into account.

It is possible to declare local arrays in Fortran subprograms, but this feature is rarely used. Typically, all arrays are declared (and dimensioned) in the main program and then passed on to the subprograms as needed.

12.1 Variable length arrays

A basic vector operation is the *saxpy* operation. This calculates the expression

```
1       y := alpha*x + y
```

where alpha is a scalar but x and y are vectors. Here is a simple subroutine for this:

```
1       subroutine saxpy (n, alpha, x, y)
2       implicit none
3       integer n
4       real alpha, x(*), y(*)
5  c
6  c Saxpy: Compute y := alpha*x + y,
7  c where x and y are vectors of length n (at least).
8  c
9  c Local variables
10      integer i
11 c
12      do 10 i = 1, n
13         y(i) = alpha*x(i) + y(i)
14    10 continue
```

```
15 c
16       return
17       end
```

The only new feature here is the use of the asterisk in the declarations *x(*)* and *y(*)*. This notation says that x and y are arrays of arbitrary length. The advantage of this is that we can use the same subroutine for all vector lengths. Recall that since Fortran is based on call-by-reference, no additional space is allocated but rather the subroutine works directly on the array elements from the calling routine/program. It is the responsibility of the programmer to make sure that the vectors x and y really have been declared to have length n or more in some other program unit. A common error in Fortran 77 occurs when you try to access out-of-bounds array elements.

We could also have declared the arrays like this:

```
1       real x(n), y(n)
```

Most programmers prefer to use the asterisk notation to emphasize that the "real array length" is unknown. Some old Fortran 77 programs may declare variable length arrays like this:

```
1       real x(1), y(1)
```

This is legal syntax even if the array lengths are greater than one! But this is poor programming style and is strongly discouraged.

12.2 Passing subsections of arrays

Next we want to write a subroutine for matrix-vector multiplication. There are two basic ways to do this, either by using inner products or saxpy operations. Let us be modular and re-use the saxpy code from the previous section. A simple code is given below.

```
1       subroutine matvec (m, n, A, lda, x, y)
2       implicit none
3       integer m, n, lda
4       real x(*), y(*), A(lda,*)
5  c
6  c Compute y = A*x, where A is m by n and stored in an array
7  c with leading dimension lda.
8  c
9  c Local variables
10      integer i, j
11
12 c Initialize y
13      do 10 i = 1, m
14         y(i) = 0.0
15    10 continue
16
17 c Matrix-vector product by saxpy on columns in A.
```

```
18 c Notice that the length of each column of A is m, not n!
19       do 20 j = 1, n
20          call saxpy (m, x(j), A(1,j), y)
21    20 continue
22
23       return
24       end
```

There are several important things to note here. First, note that even if we have written the code as general as possible to allow for arbitrary dimensions m and n, we still need to specify the leading dimension of the matrix A. The variable length declaration (*) can only be used for the *last* dimension of an array! The reason for this is the way Fortran 77 stores multidimensional arrays (see the section on arrays).

When we compute y = A*x by saxpy operations, we need to access columns of A. The j'th column of A is A(1:m,j). However, in Fortran 77 we cannot use such subarray syntax (but it is encouraged in Fortran 90!). So instead we provide a *pointer* to the first element in the column, which is A(1,j) (it is not really a pointer, but it may be helpful to think of it as if it were). We know that the next memory locations will contain the succeeding array elements in this column. The saxpy subroutine will treat A(1,j) as the first element of a vector, and does not know that this vector happens to be a column of a matrix.

Finally, note that we have stuck to the convention that matrices have m rows and n columns. The index i is used as a row index (1 to m) while the index j is used as a column index (1 to n). Most Fortran programs handling linear algebra use this notation and it makes it a lot easier to read the code!

12.3 Different dimensions

Sometimes it can be beneficial to treat a 1-dimensional array as a 2-dimensional array and vice versa. This is fairly simple to do in Fortran 77, some will say it is *too* easy!

Let us look at a very simple example. Another basic vector operation is *scaling*, i.e. multiplying each element in a vector by the same constant. Here is a subroutine for this:

```
1        subroutine scale(n, alpha, x)
2        implicit none
3        integer n
4        real alpha, x(*)
5  c
6  c Local variables
7        integer i
8
9        do 10 i = 1, n
10          x(i) = alpha * x(i)
```

```
   10 continue

      return
      end
```

Now suppose we have a m by n matrix we want to scale. Instead of writing a new subroutine for this, we can simply treat the matrix as a vector and use the subroutine `scale`. A simple version is given first:

```
      integer m, n
      parameter (m=10, n=20)
      real alpha, A(m,n)

c Some statements here define A...

c Now we want to scale A
      call scale(m*n, alpha, A)
```

Note that this example works because we assume the declared dimension of A equals the actual dimension of the matrix stored in A. This does not hold in general. Often the leading dimension lda is different from the actual dimension m, and great care must be taken to handle this correctly. Here is a more robust subroutine for scaling a matrix that uses the subroutine `scale`:

```
      subroutine mscale(m, n, alpha, A, lda)
      implicit none
      integer m, n, lda
      real alpha, A(lda,*)
c
c Local variables
      integer j

      do 10 j = 1, n
         call scale(m, alpha, A(1,j) )
   10 continue

      return
      end
```

This version works even when m is not equal to lda since we scale one column at a time and only process the m first elements of each column (leaving the rest untouched).

To see the difference, the following program stores a 2 by 2 matrix (all ones) in a 3 by 2 array, so that $m = 2$ but $lda = 3$, and scales it by 10, first with `mscale` and then with the "simple version" `scale(m*n, alpha, A)`. The program must be compiled together with the subroutines `scale` and `mscale` above.

```
      program sclmat
      implicit none
      integer lda, m, n, i, j
```

```
4       parameter (lda=3, m=2, n=2)
5       real alpha, A(lda,n)
6
7       alpha = 10.0
8
9       call fill(A, lda, n)
10      call mscale(m, n, alpha, A, lda)
11      write(*,*) 'With mscale:'
12      do 10 i = 1, lda
13         write(*,*) (A(i,j), j=1,n)
14   10 continue
15
16      call fill(A, lda, n)
17      call scale(m*n, alpha, A)
18      write(*,*) 'With scale(m*n, alpha, A):'
19      do 20 i = 1, lda
20         write(*,*) (A(i,j), j=1,n)
21   20 continue
22
23      stop
24      end
25
26      subroutine fill(A, lda, n)
27      implicit none
28      integer lda, n, i, j
29      real A(lda,n)
30
31 c Set all the elements of the array (not only the matrix) to 1
32      do 20 j = 1, n
33         do 10 i = 1, lda
34            A(i,j) = 1.0
35   10    continue
36   20 continue
37
38      return
39      end
```

Output:

```
With mscale:
  10.0000000       10.0000000
  10.0000000       10.0000000
  1.00000000       1.00000000
With scale(m*n, alpha, A):
  10.0000000       10.0000000
  10.0000000       1.00000000
  10.0000000       1.00000000
```

The third row is not part of the matrix. `mscale` correctly leaves it alone. `scale(m*n, alpha, A)`, on the other hand, simply scales the first $m \cdot n = 4$ elements in memory. Since arrays are stored by columns, these are A(1,1), A(2,1), A(3,1) and A(1,2): one element outside the matrix is changed, and one element of the matrix, A(2,2), is not.

12.4 Exercises

Solutions are in the directory `code/exercises/ch12` that accompanies this tutorial.

1. Write a real function `vnorm(n, x)` which returns the Euclidean norm $\sqrt{x_1^2 + \cdots + x_n^2}$ of a vector. Use it to compute the norm of each column of an $m \times n$ matrix stored in an array whose leading dimension is larger than m. (Hint: pass `A(1,j)`.)

2. Write a subroutine `transp(m, n, A, lda, B, ldb)` that stores the transpose of the $m \times n$ matrix `A` in `B`. Test it with arrays whose leading dimensions are larger than the matrices they contain.

3. Write a subroutine `matvc2(m, n, A, lda, x, y)` that computes $y = Ax$ by *inner products*: $y_i = \sum_{j=1}^{n} a_{ij} x_j$. Check that it gives the same result as `matvec`. Why can `matvec` pass a column of `A` to `saxpy` as if it were a vector, while a row cannot be passed in the same way?

4. In the program `sclmat`, change the leading dimension to `lda=4` (keeping $m = n = 2$). Without running the program, which elements of `A` does `call scale(m*n, alpha, A)` change now? Check your answer by running the program.

Chapter 13

Common blocks and include files

Fortran 77 has no *global* variables, i.e. variables that are shared among several program units (subroutines). The only way to pass information between subroutines we have seen so far is to use the subroutine parameter list. Sometimes this is inconvenient, e.g., when many subroutines share a large set of parameters. In such cases one can use a *common block*. This is a way to specify that certain variables should be shared among certain subroutines. But in general, the use of common blocks should be minimized.

13.1 Common blocks example

Suppose you have two parameters alpha and beta that many of your subroutines need. The following example shows how it can be done using common blocks.

```
1       program main
2       some declarations
3       real alpha, beta
4       common /coeff/ alpha, beta
5
6       statements
7       stop
8       end
9
10      subroutine sub1 (some arguments)
11      declarations of arguments
12      real alpha, beta
13      common /coeff/ alpha, beta
14
15      statements
16      return
17      end
18
```

```
19      subroutine sub2 (some arguments)
20      declarations of arguments
21      real alpha, beta
22      common /coeff/ alpha, beta
23
24      statements
25      return
26      end
```

Here we define a common block with the name `coeff`. The contents of the common block are the two variables `alpha` and `beta`. A common block can contain as many variables as you like. They do not need to all have the same type. Every subroutine that wants to use any of the variables in the common block has to declare the whole block.

Note that in this example we could easily have avoided common blocks by passing alpha and beta as parameters (arguments). A good rule is to try to avoid common blocks if possible. However, there are a few cases where there is no other solution.

13.2 Common blocks syntax

```
1       common / name / list-of-variables
```

You should know that

- The `common` statement should appear together with the variable declarations, before the executable statements.
- Different common blocks must have different names (just like variables).
- A variable *cannot* belong to more than one common block.
- The variables in a common block do not need to have the same names each place they occur (although it is a good idea to do so), but they must be listed in the same order and have the same type and size.

To illustrate this, look at the following continuation of our example:

```
1       subroutine sub3 (some arguments)
2       declarations of arguments
3       real a, b
4       common /coeff/ a, b
5
6       statements
7       return
8       end
```

This declaration is equivalent to the previous version that used `alpha` and `beta`. It is recommended that you always use the same variable names for the same common block to avoid confusion. Here is a dreadful example:

```
      subroutine sub4 (some arguments)
      declarations of arguments
      real alpha, beta
      common /coeff/ beta, alpha

      statements
      return
      end
```

Now alpha is the beta from the main program and vice versa. If you see something like this, it is probably a mistake. Such bugs are very hard to find, because the compiler has no way to detect them: a common block is just a piece of memory, and each program unit decides for itself what names to give to its parts.

The following complete program puts the three versions side by side:

```
      program coeffs
      implicit none
      real alpha, beta
      common /coeff/ alpha, beta

      alpha = 1.0
      beta = 2.0
      call sub1
      call sub3
      call sub4

      stop
      end

      subroutine sub1
      implicit none
      real alpha, beta
      common /coeff/ alpha, beta

      write(*,*) 'sub1: alpha =', alpha, '  beta =', beta
      return
      end

      subroutine sub3
      implicit none
      real a, b
      common /coeff/ a, b

      write(*,*) 'sub3: a     =', a, '  b    =', b
      return
      end

      subroutine sub4
```

```
34      implicit none
35      real alpha, beta
36      common /coeff/ beta, alpha
37
38      write(*,*) 'sub4: alpha =', alpha, '  beta =', beta
39      return
40      end
```

Output:

```
sub1: alpha =   1.00000000       beta =   2.00000000
sub3: a     =   1.00000000       b    =   2.00000000
sub4: alpha =   2.00000000       beta =   1.00000000
```

The values in `sub4` are swapped, and nothing warns us about it.

13.3 Arrays in common blocks

Common blocks can include arrays, too. But again, this is not recommended. The major reason is flexibility. An example shows why this is such a bad idea. Suppose we have the following declarations in the main program:

```
1       program main
2       integer nmax
3       parameter (nmax=20)
4       integer n
5       real A(nmax, nmax)
6       common /matrix/ A, n
```

This common block contains first all the elements of A, then the integer n. Now assume you want to use the matrix A in some subroutines. Then you have to include the same declarations in all these subroutines, e.g.

```
1       subroutine sub1 (...)
2       integer nmax
3       parameter (nmax=20)
4       integer n
5       real A(nmax, nmax)
6       common /matrix/ A, n
```

Arrays with variable dimensions cannot appear in common blocks, thus the value of *nmax* has to be exactly the same as in the main program. Recall that the size of a matrix has to be known at compile time, hence nmax has to be defined in a parameter statement.

This example shows there is usually nothing to gain by putting arrays in common blocks. Hence the preferred method in Fortran 77 is to pass arrays as arguments to subroutines (along with the leading dimensions).

13.4 Blank common blocks

Fortran 77 also allows for common blocks without names, called *blank common blocks*. The syntax is simply:

```
common list-of-variables
```

For example:

```
common x, y, z, count
```

Blank common blocks have some special properties:

- There is only one blank common block in the whole program. (If a program unit has several `common` statements without a name, their lists are simply joined.)
- The variables in the blank common block cannot be initialized by `DATA` statements, not even in a `block data` unit
- Unlike named common blocks, the blank common block can have different sizes in different program units
- The values in the blank common block never become undefined, so it never needs to be saved

Here's an example of using a blank common block:

```
program blankc
implicit none
real x, y
integer n
common x, y, n

x = 1.0
y = 2.0
n = 3
call sub

write(*,*) 'After sub: x =', x, ' y =', y, ' n =', n

stop
end

subroutine sub
implicit none
real a, b
integer i
common a, b, i

write(*,*) 'In sub: a =', a, ' b =', b, ' i =', i
a = 10.0
b = 20.0
```

```
26      i = 30
27
28      return
29      end
```

In this example, even though different variable names are used, `x` and `a`, `y` and `b`, and `n` and `i` refer to the same memory locations. The program will output:

```
In sub: a =   1.00000000      b =   2.00000000      i =           3
After sub: x =   10.0000000      y =   20.0000000      n =
    30
```

Blank common blocks are generally less preferred than named common blocks because they provide less structure and are more prone to errors.

13.5 The `INCLUDE` statement

When using common blocks in multiple program units, it's tedious and error-prone to repeat the same common block declarations. The `INCLUDE` statement is the usual solution to this problem. Strictly speaking, it is *not* part of the Fortran 77 standard: it was a very widespread extension (defined in the U.S. military standard MIL-STD-1753 of 1978) and only became standard in Fortran 90. In practice every Fortran 77 compiler supports it.

The `INCLUDE` statement allows you to insert the contents of an external file directly into your program at compile time. The syntax is:

```
1       INCLUDE 'filename'
```

The `INCLUDE` statement can appear anywhere in your program where the included text would be valid. It's particularly useful for common block declarations, parameter definitions, and other specifications that need to be consistent across multiple program units.

Here's an example of how to use the `INCLUDE` statement with common blocks:

First, create a file called `params.inc` containing:

```
1       real alpha, beta
2       common /coeff/ alpha, beta
```

Then, in your Fortran program units:

```
1       program incl
2       implicit none
3       real x
4       INCLUDE 'params.inc'
5
6       alpha = 1.0
7       beta = 2.0
8       call sub1(x)
9       write(*,*) 'x =', x, ' alpha =', alpha, ' beta =', beta
```

```
10
11      stop
12      end
13
14      subroutine sub1(x)
15      implicit none
16      real x
17      INCLUDE 'params.inc'
18
19      x = alpha + beta
20      alpha = 3.0
21
22      return
23      end
```

Output:

```
 x =   3.00000000      alpha =   3.00000000      beta =   2.00000000
```

The compiler looks for `params.inc` in the directory of the source file; with gfortran you can name other directories with the option `-I`, e.g. `gfortran -I../include main.f`.

This ensures that the common block is consistently defined across all program units. If you need to change the common block structure, you only need to modify the include file once, rather than updating each occurrence in your program.

13.5.1 Benefits of INCLUDE

The `INCLUDE` statement offers several advantages:

- Reduces code duplication
- Ensures consistency across program units
- Makes maintenance easier - changes in one place affect all included instances
- Can be used for more than just common blocks (parameter definitions, type declarations, etc.)

13.5.2 Limitations of INCLUDE

There are some limitations to be aware of:

- As explained above, `INCLUDE` is an extension to Fortran 77, although a universal one
- The `INCLUDE` line must be the only statement on its line, and it cannot be continued

- Inclusion is unconditional: there is no way to include a file only under certain conditions (for that, some compilers offer a C-style preprocessor with `#include` and `#ifdef`)
- How the file name is interpreted (directories, upper/lower case) depends on the compiler and the operating system

13.5.3 Best practices for `INCLUDE` files

Some recommendations for using include files:

- Use a standardized file extension (like `.inc`) for include files
- Keep include files small and focused on a single purpose
- Document the purpose and contents of each include file
- Group related variables in the same include file
- Consider using different include files for different common blocks

13.6 Exercises

Solutions are in the directory `code/exercises/ch13` that accompanies this tutorial.

1. Write a program that reads the coefficients a, b, c of the polynomial $f(x) = ax^2 + bx + c$ and prints $f(x)$ for $x = 0, 1, \ldots, 5$. The coefficients should be in a common block, declared in an include file `poly.inc`, so that the real function `f` has only one argument, `x`.

2. What does the following program print? Why?

```
      program swapc
      implicit none
      real x, y
      common /c/ x, y

      x = 1.0
      y = 2.0
      call addten
      write (*,*) 'x =', x, '  y =', y

      stop
      end

      subroutine addten
      implicit none
      real x, y
      common /c/ y, x
```

```
18
19        x = x + 10.0
20
21        return
22        end
```

3. Write a subroutine `tick` (without arguments) that counts how many times it has been called, in a variable `ncalls` of a common block `/stats/`. The main program should call `tick` five times in a loop and then print `ncalls`. How must the main program give `ncalls` its initial value?

4. Rewrite the program `common_array.f` that accompanies this tutorial (the example of the section "Arrays in common blocks") so that the declarations of `nmax`, `n`, `A` and the common block `/matrix/` are in an include file `matrix.inc`.

Chapter 14

Data and Block Data Statements

14.1 The data statement

The data statement is another way to input data that are known at the time when the program is written. It is similar to the assignment statement. The syntax is:

```
1       data list-of-variables/ list-of-values/, ...
```

where the three dots means that this pattern can be repeated. Here is an example:

```
1       data m/10/, n/20/, x/2.5/, y/2.5/
```

We could also have written this

```
1       data m,n/10,20/, x,y/2*2.5/
```

We could have accomplished the same thing by the assignments

```
1       m = 10
2       n = 20
3       x = 2.5
4       y = 2.5
```

The data statement is more compact and therefore often more convenient. Notice especially the shorthand notation for assigning identical values repeatedly.

There is, however, an important difference. An assignment is executed every time the program reaches it, while the data statement is not executed at all: it only specifies the *initial* values that the variables have when the program starts. (Usually the compiler simply stores the values in the executable file.) The data statement can therefore appear anywhere among the declarations, and it is often used in subroutines to give a variable an initial

value that must be set only once, as in the `counter` subroutine of Chapter 11 (remember to `save` such variables).

The data statement can also be used to initialize arrays (vectors, matrices). This example shows how to make sure a matrix is all zeros when the program starts:

```
      real A(10,20)
      data A/ 200 * 0.0/
```

Some compilers will automatically initialize arrays like this but not all, so if you rely on array elements to be zero it is a good idea to follow this example. Of course you can initialize arrays to other values than zero. You may even initialize individual elements:

```
      data A(1,1)/ 12.5/, A(2,1)/ -33.3/, A(2,2)/ 1.0/
```

Or you can list all the elements for small arrays like this:

```
      integer v(5)
      real B(2,2)
      data v/10,20,30,40,50/, B/1.0,-3.7,4.3,0.0/
```

The values for two-dimensional arrays will be assigned in column-first order as usual, i.e. B(1,1) = 1.0, B(2,1) = −3.7, B(1,2) = 4.3 and B(2,2) = 0.0. If you print `B` with `write(*,*)`, gfortran shows

```
 1.00000000      -3.70000005       4.30000019       0.00000000
```

The "wrong" last digits are not an error of the data statement: −3.7 and 4.3 cannot be represented exactly in binary (see Chapter 5), and list-directed output shows enough digits to make the rounding visible.

14.2 The block data statement

In standard Fortran 77, the data statement cannot be used for variables contained in a named common block, except in a special program unit called `block data`. It is not really a subroutine, but it looks a bit similar because it is given as a separate program unit. Here is an example, together with a main program that uses the common block:

```
      program bdata
      implicit none
      integer nmax
      parameter (nmax=20)
      real v(nmax), alpha, beta
      common /vector/v,alpha,beta

      write(*,*) 'v(1) =', v(1), '  v(20) =', v(20)
      write(*,*) 'alpha =', alpha, '  beta =', beta

      stop
```

```
12      end
13
14      block data vecini
15      implicit none
16      integer nmax
17      parameter (nmax=20)
18      real v(nmax), alpha, beta
19      common /vector/v,alpha,beta
20      data v/20*100.0/, alpha/3.14/, beta/2.71/
21      end
```

Output:

```
v(1) =   100.000000      v(20) =   100.000000
alpha =   3.14000010      beta =   2.71000004
```

Just as the data statement, a block data unit contains no executable statements; it only defines initial values, which the variables have when the program starts. The position of the block data unit in the source code is irrelevant (as long as it is not nested inside the main program or a subprogram).

It is a good idea to give the block data unit a name (here `vecini`). If the block data unit is placed in a library, the linker may not include it, because no other unit refers to it, and the common block would silently remain uninitialized. Writing `external vecini` in the main program forces the linker to include it.

14.3 Exercises

Solutions are in the directory `code/exercises/ch14` that accompanies this tutorial.

1. Write a program that reads a date (month and day) and prints the day of the year, for a year that is not a leap year. Give the number of days of each month with a data statement: `integer mdays(12)` and `data mdays /31, 28, .../`.

2. After the declarations `real B(2,3)` and `data B /2*1.0, 3*2.0, 3.0/`, what is the value of each element of `B`? Answer first without a computer, and then check with a program that prints `B` row by row.

3. Write two subroutines, `count1` and `count2`, that add 1 to a local integer variable `n` and print it. In `count1`, `n` starts at 0 with `save n` and `data n /0/`; in `count2`, with the assignment `n = 0`. What does each one print if it is called three times? Why?

4. Put the speed of light $c = 299\,792\,458$ m/s, the Planck constant $h = 6.62607015 \cdot 10^{-34}$ J s and the elementary charge $e = 1.602176634 \cdot$

10^{-19} C in a named common block of double precision variables, initialized in a named block data unit. Write a main program that prints them, and the energy hc/λ of a photon of wavelength $\lambda = 500$ nm. Use an `external` statement for the block data unit.

Chapter 15

File I/O

So far we have assumed that the input/output has been to the standard input or the standard output. It is also possible to read from or write to *files* which are stored on some external storage device, typically a disk. In Fortran each file is associated with a *unit number*, a non-negative integer. The allowed range depends on the compiler, but 1 to 99 is safe everywhere. Some unit numbers are traditionally preconnected: on almost all systems 5 is standard input and 6 is standard output (and often 0 is standard error), so do not use them for your own files.

15.1 Opening and closing a file

Before you can use a file you have to *open* it. The command is

```
1    open (list-of-specifiers)
```

where the most common specifiers are:

```
1    [UNIT=]   u
2    IOSTAT=   ios
3    ERR=      err
4    FILE=     fname
5    STATUS=   sta
6    ACCESS=   acc
7    FORM=     frm
8    RECL=     rl
```

The unit number *u* is a number in the range 1-99 that denotes this file (the programmer may chose any number but he/she has to make sure it is unique).

ios is the I/O status identifier and should be an integer variable. Upon return, *ios* is zero if the statement was successful and returns a non-zero value otherwise.

err is a label which the program will jump to if there is an error.

fname is a character string denoting the file name.

sta is a character string that has to be either NEW, OLD, SCRATCH or UNKNOWN. It shows the prior status of the file: OLD means the file must already exist, NEW means it must not exist yet (it is then created). A scratch file is a file that is created when opened and deleted when closed (or the program ends). UNKNOWN, the default, leaves the decision to the system; in practice it opens the file if it exists and creates it otherwise.

acc must be either SEQUENTIAL or DIRECT. The default is SEQUENTIAL.

frm must be either FORMATTED or UNFORMATTED. The default is FORMATTED for sequential files and UNFORMATTED for direct access files.

rl specifies the length of each record in a direct-access file.

For more details on these specifiers, see a good Fortran 77 book.

After a file has been opened, you can access it by read and write statements. When you are done with the file, it should be closed by the statement

```
      close ([UNIT=]u[,IOSTAT=ios,ERR=err,STATUS=sta])
```

where, as usual, the parameters in brackets are optional.

In this case *sta* is a character string which can be KEEP (the default) or DELETE.

15.1.1 Basic File Opening and Closing Example

Here's a simple example demonstrating the basic concepts of opening, writing to, and closing a file:

```
      program basic_file
      implicit none
      integer unit

c Basic file operations
      unit = 10

c Open the file for writing
      open (unit, file='output.txt', status='unknown')

      write (unit,*) 'Hello, file!'
      write (unit,*) 'This is line 2'

c Close the file
      close (unit)

      write (*,*) 'File created successfully'

      stop
      end
```

After running the program, the file `output.txt` contains the two lines. Some remarks:

- We used `status='unknown'`, so the program can be run again and again. With `status='new'`, the second run would stop with an error like *Cannot open file 'output.txt': File exists*. Use NEW only when overwriting an existing file would be a mistake.
- With `status='unknown'`, an existing file is opened positioned at its beginning, so writing to it replaces its old contents.
- The variable that holds the unit number is called `unit`. This is legal, since Fortran has no reserved words, but some programmers prefer a name such as `iu` to avoid confusion with the `unit=` specifier.

15.2 Read and write revisited

The only necessary change from our previous simplified read/write statements, is that the unit number must be specified. But frequently one wants to add more specifiers. Here is how:

```
1       read ([UNIT=]u, [FMT=]fmt, IOSTAT=ios, ERR=err, END=s)
2       write([UNIT=]u, [FMT=]fmt, IOSTAT=ios, ERR=err)
```

where most of the specifiers have been described above. The END=s specifier, which is only allowed in `read`, defines which statement label the program jumps to if it reaches end-of-file.

15.3 Error Handling

Proper error handling is crucial when working with files. Fortran 77 provides two main mechanisms for error detection: the IOSTAT parameter and the ERR parameter.

15.3.1 Using IOSTAT for Error Detection

The IOSTAT parameter returns an integer status code. Zero indicates success, while non-zero values indicate various error conditions:

```
1       program iostat_example
2       implicit none
3       integer unit, ios
4       character*20 filename
5
6       unit = 15
7       filename = 'nonexistent.dat'
8
9   c Try to open a file that may not exist
10      open (unit, file=filename, status='old', iostat=ios)
11
12      if (ios .eq. 0) then
```

```
         write (*,*) 'File opened successfully'
         close (unit)
      else
         write (*,*) 'Error opening file: ', filename
         write (*,*) 'Error code: ', ios
      endif

      stop
      end
```

15.3.2 Using ERR for Error Handling

The ERR parameter specifies a statement label to jump to when an error occurs:

```
      program err_example
      implicit none
      integer unit
      character*20 filename

      unit = 12
      filename = 'missing.dat'

c Try to open file, jump to error handler if it fails
      open (unit, file=filename, status='old', err=100)

      write (*,*) 'File opened successfully'
      close (unit)
      goto 200

c Error handler
  100 write (*,*) 'Cannot open file: ', filename
      write (*,*) 'Please check if file exists'

  200 stop
      end
```

15.4 File STATUS Parameter

The STATUS parameter controls how Fortran treats the file when opening it. Besides UNKNOWN, the three main options are NEW, OLD, and SCRATCH:

```
      program status_examples
      implicit none
      integer unit1, unit2, unit3
      character*40 line

c Example 1: Create a NEW file
      unit1 = 11
      open (unit1, file='newfile.txt', status='new')
```

```
      write (unit1,'(A)') 'This is a new file'
      close (unit1)

c Example 2: Open an OLD (existing) file, read back the line
c we wrote, and delete the file when closing it
      unit2 = 12
      open (unit2, file='newfile.txt', status='old')
      read (unit2,'(A)') line
      write (*,*) 'Read from newfile.txt: ', line
      close (unit2, status='delete')

c Example 3: Create a SCRATCH file (temporary)
      unit3 = 13
      open (unit3, status='scratch')
      write (unit3,*) 'This is temporary data'
      write (unit3,*) 'It will be deleted when closed'
      close (unit3)  ! File is automatically deleted

      write (*,*) 'All file operations completed'

      stop
      end
```

Output:

```
Read from newfile.txt: This is a new file
All file operations completed
```

Because the second `close` uses `status='delete'`, the file `newfile.txt` no longer exists when the program ends, and the program can be run again. Try changing `'delete'` to `'keep'` and running the program twice: the second time the `open` with `status='new'` fails, because the file already exists.

The format `'(A)'` used here reads or writes a whole line as text; it is explained in Chapter 17.

15.5 File ACCESS Methods

Fortran 77 supports two types of file access: sequential and direct access.

15.5.1 Sequential Access

Sequential access processes records in order from beginning to end:

```
      program access_examples
      implicit none
      integer unit, i
      real data(5)

c Sequential access (default)
      unit = 20
      open (unit, file='sequential.dat', access='sequential')
```

```
9
10        do 10 i = 1, 5
11           data(i) = real(i) * 1.5
12           write (unit,*) data(i)
13     10 continue
14        close (unit)
15
16  c Read back sequentially
17        open (unit, file='sequential.dat', access='sequential')
18        do 20 i = 1, 5
19           read (unit,*) data(i)
20           write (*,*) 'Read value:', data(i)
21     20 continue
22        close (unit)
23
24        stop
25        end
```

15.5.2 Direct Access

Direct access allows reading and writing records in any order. Each record must have the same length:

```
1         program direct_access
2         implicit none
3         integer unit, record_num
4         real value
5
6         unit = 25
7
8   c Create direct access file with fixed record length
9         open (unit, file='direct.dat', access='direct', recl=20)
10
11  c Write records in any order
12        write (unit, rec=3) 30.0
13        write (unit, rec=1) 10.0
14        write (unit, rec=5) 50.0
15        write (unit, rec=2) 20.0
16        write (unit, rec=4) 40.0
17
18  c Read records in any order
19        do 10 record_num = 1, 5
20           read (unit, rec=record_num) value
21           write (*,*) 'Record', record_num, ':', value
22     10 continue
23
24        close (unit)
25
26        stop
27        end
```

15.6 File FORM Parameter

The FORM parameter specifies whether the file is formatted (human-readable) or unformatted (binary):

```
      program form_examples
      implicit none
      integer unit, i
      real numbers(3)

      numbers(1) = 3.14159
      numbers(2) = 2.71828
      numbers(3) = 1.41421

c Write formatted file (human readable)
      unit = 30
      open (unit, file='formatted.txt', form='formatted')
      write (unit,100) (numbers(i), i=1,3)
  100 format (3f10.5)
      close (unit)

c Write unformatted file (binary, more efficient)
      unit = 31
      open (unit, file='unformatted.dat', form='unformatted')
      write (unit) (numbers(i), i=1,3)
      close (unit)

c Read back unformatted file
      open (unit, file='unformatted.dat', form='unformatted')
      read (unit) (numbers(i), i=1,3)
      close (unit)

      write (*,*) 'Values from unformatted file:'
      write (*,*) (numbers(i), i=1,3)

      stop
      end
```

15.7 End-of-File Detection

The END parameter allows programs to detect when they reach the end of a file:

```
      program end_example
      implicit none
      integer unit, count
      real value

      unit = 40
      count = 0

c First, create a data file
```

```
      open (unit, file='data.txt')
      write (unit,*) 1.0
      write (unit,*) 2.5
      write (unit,*) 3.7
      write (unit,*) 4.2
      close (unit)

c Now read until end of file
      open (unit, file='data.txt', status='old')

   10 read (unit, *, end=20) value
      count = count + 1
      write (*,*) 'Value', count, ':', value
      goto 10

   20 write (*,*) 'End of file reached. Total values:', count
      close (unit)

      stop
      end
```

15.8 Comprehensive Error Handling Example

Here's a more sophisticated example that demonstrates comprehensive error handling:

```
      program complete_error_handling
      implicit none
      integer unit, ios, count
      real value, sum
      character*30 filename

      unit = 50
      filename = 'numbers.dat'
      count = 0
      sum = 0.0

c Open file with complete error checking
      open (unit, file=filename, status='old', iostat=ios, err=100)

      write (*,*) 'Reading from file: ', filename

c Read data with end-of-file detection
   10 read (unit, *, iostat=ios, end=30, err=200) value
      count = count + 1
      sum = sum + value
      write (*,*) 'Read value', count, ':', value
      goto 10

c Normal end of file
   30 write (*,*) 'Successfully read', count, 'values'
      write (*,*) 'Sum =', sum
```

```
      write (*,*) 'Average =', sum / real(count)
      close (unit)
      goto 999

c File open error
  100 write (*,*) 'Error: Cannot open file ', filename
      write (*,*) 'Error code:', ios
      goto 999

c Read error
  200 write (*,*) 'Error reading from file at record', count+1
      write (*,*) 'Error code:', ios
      close (unit)

  999 stop
      end
```

15.9 File Existence Checking

The INQUIRE statement can be used to check if a file exists before attempting to open it:

```
      program file_exists
      implicit none
      character*20 filename
      logical exists
      integer unit

      filename = 'test.dat'
      unit = 60

c Check if file exists
      inquire (file=filename, exist=exists)

      if (exists) then
         write (*,*) 'File ', filename, ' exists'
         open (unit, file=filename, status='old')
         write (*,*) 'File opened for reading'
      else
         write (*,*) 'File ', filename, ' does not exist'
         write (*,*) 'Creating new file...'
         open (unit, file=filename, status='new')
         write (unit,*) 'This is a new file'
         write (*,*) 'File created and written'
      endif

      close (unit)

      stop
      end
```

15.10 Appending to Files

To add data to the end of an existing file, you can use the POSITION parameter:

```
      program append_example
      implicit none
      integer unit, i

      unit = 70

c Create initial file
      open (unit, file='log.txt', status='unknown')
      write (unit,*) 'Initial entry'
      close (unit)

c Append to existing file (reopen and position at end)
      open (unit, file='log.txt', status='old', position='append')

      do 10 i = 1, 3
         write (unit,*) 'Appended entry number', i
   10 continue

      close (unit)

      write (*,*) 'Data appended to log.txt'

      stop
      end
```

Note that the `POSITION` specifier is not part of Fortran 77: it was introduced in Fortran 90. It is supported by all current compilers. In strict Fortran 77, the usual way to append is to read the file until the end-of-file condition occurs, then `backspace` over the end-of-file and write the new records.

15.11 A complete example: points in space

You are given a data file with xyz coordinates for a bunch of points. The number of points is given on the first line. The file name of the data file is *points.dat*. The format for each coordinate is known to be F10.4, i.e. each number occupies 10 columns and has 4 decimals (we'll learn about FORMAT statements in Chapter 17). For example:

```
4
    0.0000    0.0000    0.0000
    2.0000    0.0000    0.0000
    0.0000    2.0000    0.0000
    2.0000    2.0000    4.0000
```

Here is a program that reads the data into 3 arrays x, y, z, computes the *centroid* of the points (the point whose coordinates are the averages of the coordinates of all points), and finds the point farthest away from the centroid:

```
      program inpdat
c
c  This program reads n points from a data file and stores them in
c  3 arrays x, y, z. Then it computes their centroid and the point
c  farthest away from it.
c
      implicit none
      integer nmax, u
      parameter (nmax=1000, u=20)
      real x(nmax), y(nmax), z(nmax)
      real xc, yc, zc, d, dmax
      integer n, i, imax

c  Open the data file
      open (u, FILE='points.dat', STATUS='OLD')

c  Read the number of points
      read(u,*) n
      if (n .GT. nmax .OR. n .LT. 1) then
         write(*,*) 'Error: n = ', n, ' must be between 1 and ', nmax
         goto 9999
      endif

c  Loop over the data points
      do 10 i = 1, n
         read(u,100) x(i), y(i), z(i)
   10 continue
  100 format (3F10.4)

c  Close the file
      close (u)

c  Centroid: the average of the coordinates
      xc = 0.0
      yc = 0.0
      zc = 0.0
      do 20 i = 1, n
         xc = xc + x(i)
         yc = yc + y(i)
         zc = zc + z(i)
   20 continue
      xc = xc / real(n)
      yc = yc / real(n)
      zc = zc / real(n)

c  The point farthest away from the centroid
      imax = 1
      dmax = -1.0
      do 30 i = 1, n
```

```
50          d = sqrt((x(i)-xc)**2 + (y(i)-yc)**2 + (z(i)-zc)**2)
51          if (d .GT. dmax) then
52             dmax = d
53             imax = i
54          endif
55    30 continue
56
57       write(*,*) 'Number of points:', n
58       write(*,*) 'Centroid:', xc, yc, zc
59       write(*,*) 'Farthest point is number', imax, ' at distance', dmax
60
61  9999 stop
62       end
```

With the data file shown above, the output is:

```
 Number of points:           4
 Centroid:   1.00000000       1.00000000       1.00000000
 Farthest point is number           4  at distance   3.31662488
```

Notice how the program checks that `n` fits in the arrays before reading the points. Without this check, a data file with more than 1000 points would make the program write beyond the end of the arrays, which in Fortran 77 is not detected and can have unpredictable consequences.

15.12 Exercises

Solutions are in the directory `code/exercises/ch15` that accompanies this tutorial.

1. Write a program that writes a table of x and $\sin(x)$, for 11 equally spaced values of x from 0 to π, to the file `sine.dat`. Then the program should open the file again, read the table back, and print the largest value of $\sin(x)$ and the x where it occurs.

2. Write a program that asks for the name of a text file, opens it (printing a message and stopping if the file cannot be opened; use `IOSTAT`), and counts its lines. Hint: read each line into a character variable with the format `'(A)'`, and use `END=` to detect the end of the file.

3. Write a program that creates a direct-access file and writes the squares of $1, 2, \ldots, 10$ to it, record i holding i^2, in *reverse* order ($i = 10, 9, \ldots, 1$). Then the program should read back records 10, 1 and 5, in this order, print them, and delete the file. (With gfortran, `RECL` is measured in bytes, and a default integer occupies 4 bytes.)

4. Run the program `status_examples` of this chapter. Then change `status='delete'` to `status='keep'` and run it twice. What happens the second time, and why? What would happen with `status='unknown'` in the first `open`?

5. Write a program that writes the same 1000 real numbers (for example $\sqrt{1}, \ldots, \sqrt{1000}$) to a formatted file, one number per line with list-directed output, and to an unformatted file with a single write statement. Compare the sizes of the two files. Why are they so different?

6. Write a program that copies the text file `poem.dat` to a new file `poem.num`, adding the line number at the beginning of each line, and then prints the new file.

7. Write a program that reads real numbers, one per line, from a file that may contain bad data (for example a line with `abc`), using `IOSTAT` in the read statement. The program should report the number of each bad line, skip it, and print the number and the sum of the good values at the end of the file. (Recall that `IOSTAT` is positive for an error and negative at the end of the file.)

8. Write a program that counts how many times it has been run, keeping the count in the file `runs.dat`. Use `INQUIRE` to find out whether the file exists (the first time it does not).

Chapter 16

Simple I/O

An important part of any computer program is to handle input and output. In our examples so far, we have already used the two most common Fortran constructs for this: `read` and `write`. Fortran I/O can be quite complicated, so we will only describe some simpler cases in this tutorial.

16.1 Read and write

`Read` is used for input, while `write` is used for output. A simple form is

```
1       read (unit no, format no) list-of-variables
2       write(unit no, format no) list-of-variables
```

The unit number can refer to either standard input, standard output, or a file. The format number refers to a label for a format statement, which will be described in the next lesson.

It is possible to simplify these statements further by using asterisks (*) for some arguments, like we have done in most of our examples so far. This is sometimes called *list directed* read/write.

```
1       read (*,*) list-of-variables
2       write(*,*) list-of-variables
```

The first statement will read values from the standard input and assign the values to the variables in the variable list, while the second one writes to the standard output.

16.2 Examples

Here is a code segment from a Fortran program:

```
1       integer m, n
2       real x, y, z(10)
3
4       read(*,*) m, n
```

```
5       read(*,*) x, y
6       read(*,*) z
```

We give the input through standard input (possibly through a data file directed to standard input, as in `./prog < data.txt`). A data file consists of *records* according to traditional Fortran terminology; for a text file, a record is simply a line. Each `read` statement starts reading at the beginning of a new record. Within a record, the values are separated by blanks or commas. Hence a legal input to the program above would be:

```
1   -1  100
2  -1.0 1e+2
3  1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0 10.0
```

Or, we could add commas as separators:

```
1   -1, 100
2  -1.0, 1e+2
3  1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0
```

Note that Fortran 77 input is line sensitive, so it is important not to have extra input elements (fields) on a line (record). For example, if we gave the first four inputs all on one line as

```
1   -1, 100, -1.0, 1e+2
2  1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0
```

then m and n would be assigned the values -1 and 100 respectively, but the last two values would be discarded, because the next `read` starts on a new line. So x and y would be assigned the values 1.0 and 2.0, ignoring the rest of the second line. Then the third `read` finds no more input for z, and the program stops with an error:

```
Fortran runtime error: End of file
```

(In Chapter 15 we saw how to handle the end of the input with `END=` or `IOSTAT=` instead of letting the program stop.)

If there are too few inputs on a line then the next line will be read. For example

```
1   -1
2  100
3  -1.0
4  1e+2
5  1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0
```

would produce the same results as the first two examples.

16.3 Other versions

For simple list-directed I/O it is possible to use the alternate syntax

```
      read  *, list-of-variables
      print *, list-of-variables
```

which has the same meaning as the list-directed read and write statements described earlier. This version always reads/writes to standard input/output so the * corresponds to the format.

16.4 Implied do-loops

To read or write many elements of an array in a single statement, Fortran has the *implied do-loop*: a list of items, followed by a loop control as in a do statement, all between parentheses:

```
      read(*,*) (x(i), i = 1, n)
      write(*,*) (x(i), i = 1, n, 2)
```

The first statement reads `x(1)`, `x(2)`, ..., `x(n)`; since it is a single read statement, the n values can be distributed over the input lines in any way. (A do loop with a read statement inside would instead start a new line for every element.) The second statement writes the elements with odd index. Implied do-loops can be nested, and are often combined with an ordinary do loop to print a matrix row by row:

```
      program implid
      implicit none
      integer n, i, j
      real x(10), A(3,3)

      read(*,*) n
      read(*,*) (x(i), i = 1, n)
      write(*,*) 'x:', (x(i), i = 1, n)
      write(*,*) 'Odd positions:', (x(i), i = 1, n, 2)

      do 20 j = 1, 3
         do 10 i = 1, 3
            A(i,j) = real(10*i + j)
   10    continue
   20 continue
      write(*,*) 'A, row by row:'
      do 30 i = 1, 3
         write(*,*) (A(i,j), j = 1, 3)
   30 continue
      write(*,*) 'A in memory order:'
      write(*,*) ((A(i,j), i = 1, 3), j = 1, 3)
      end
```

With the input

```
5
1.5 2.5 3.5
4.5 5.5
```

the program prints (the last line is shown here broken in two)

```
x:   1.50000000       2.50000000       3.50000000       4.50000000
        5.50000000
Odd positions:   1.50000000       3.50000000       5.50000000
A, row by row:
  11.0000000       12.0000000       13.0000000
  21.0000000       22.0000000       23.0000000
  31.0000000       32.0000000       33.0000000
A in memory order:
  11.0000000       21.0000000       31.0000000       12.0000000
    22.0000000
  32.0000000       13.0000000       23.0000000       33.0000000
```

In the nested implied loop the *inner* loop (over `i`) runs fastest, so the elements come out in the order in which they are stored in memory, column by column. Writing just the name of the array, `write(*,*) A`, gives the same result.

16.5 Exercises

Solutions are in the directory `code/exercises/ch16` that accompanies this tutorial.

1. What values do the variables receive if the program

```
1       integer a, b, c, d, e
2       read (*,*) a, b
3       read (*,*) c, d
4       read (*,*) e
```

   reads the input below? Answer first without a computer.

```
1 1 2 3
2 4
3 5
4 6 7
```

2. Write a program that reads an integer $n \leq 100$ and then n integers, distributed over the input lines in any way, with an implied do-loop. The program should print their sum, and the numbers in reverse order (use an implied do-loop with a negative increment).

3. Write a program that reads a 3×3 matrix row by row, one row on each line, and prints its transpose row by row.

4. Run the program `checks/eof_extra.f` that accompanies this tutorial with the input `checks/eof_extra.in` (the case with the first four values on one line). Explain the error message. Which values would `x` and `y` receive?

Chapter 17

Format statements

So far we have mostly used *free format* input/output. This uses a set of default rules for how to input and output values of different types (integers, reals, characters, etc.). Often the programmer wants to specify some particular input or output format, e.g., how many decimals places in real numbers. For this purpose Fortran 77 has the *format* statement. The same format statements are used for both input and output.

17.1 Syntax

```
1       write(*, label) list-of-variables
2  label format format-code
```

A simple example demonstrates how this works. Say you have an integer variable you want to print in a field 4 characters wide and a real number you want to print in fixed point notation with 3 decimal places.

```
1       write(*, 900) i, x
2   900 format (I4,F8.3)
```

The format label 900 is chosen somewhat arbitrarily, but it is common practice to number format statements with higher numbers than the control flow labels. After the keyword `format` follows the format codes enclosed in parenthesis. The code I4 stands for an integer with width four, while F8.3 means that the number should be printed using fixed point notation with field width 8 and 3 decimal places.

The format statement may be located anywhere within the program unit. There are two programming styles: Either the format statement follows directly after the read/write statement, or all the format statements are grouped together at the end of the (sub-)program.

17.2 Common format codes

The most common format code letters are:

```
  A - text string
  D - double precision numbers, exponent notation
  E - real numbers, exponent notation
  F - real numbers, fixed point format
  G - real numbers, F or E notation depending on the value
  I - integer
  L - logical (T or F)
  X - horizontal skip (space)
  / - vertical skip (newline)
```

The format code F has the general form F*w.d* where *w* is an integer constant denoting the field width (the total number of characters, including sign and decimal point) and *d* is an integer constant denoting the number of digits after the decimal point. E*w.d* and D*w.d* are similar, but the number is written as a fraction between 0.1 and 1 with *d* digits, followed by an exponent; e.g. 0.025 written with E10.3 is `0.250E-01`.

For integers only the field width is specified, so the syntax is `I`*w*. The variant `I`*w.m* writes at least *m* digits, with leading zeros if necessary (`I2.2` writes 7 as `07`). Similarly, character strings can be specified as `A`*w*, but the field width is often dropped: plain `A` uses the length of the string.

If a number or string does not fill up the entire field width, spaces will be added. Numbers are adjusted to the right of the field; strings written with `A`*w* are adjusted to the left. If a number does *not fit* in the field, Fortran does not widen the field: it fills it with asterisks instead, as we will see below.

For horizontal spacing, the *n*`X` code is often used. This means *n* horizontal spaces. For vertical spacing (newlines), use the code `/`. Each slash corresponds to one newline. Note that each read or write statement by default ends with a newline (here Fortran differs from C).

17.3 Some examples

This program

```
      program fmtex
      implicit none
      real x
      integer i

      x = 0.025
      i = 42
      write(*,100) 'x=', x
  100 format (A,F8.4)
      write(*,110) 'x=', x
  110 format (A,F5.3)
      write(*,120) 'x=', x
```

```
13    120 format (A,E12.4)
14        write(*,130) 'x=', x
15    130 format (A,E8.1)
16        write(*,140) 'i=', i, 'x=', 123.456
17    140 format (A,I4,2X,A,F6.2)
18        write(*,150) 'i=', i, 'x=', 123.456
19    150 format (A,I1,2X,A,F4.2)
20
21        stop
22        end
```

produces the following output when we run it:

```
x=  0.0250
x=0.025
x=  0.2500E-01
x= 0.3E-01
i=  42  x=123.46
i=*  x=****
```

Note how blanks are automatically padded on the left, and how the numbers are rounded to the number of decimals asked for (0.025 becomes 0.3E-01 with one digit, and 123.456 becomes 123.46). The last line shows what happens when a field is too narrow: 42 does not fit in one column and 123.456 does not fit in four, so the fields are filled with asterisks. If you see asterisks in the output of a Fortran program, a format is too narrow.

Also notice that, unlike list-directed output, formatted output does not start each line with a blank. In old programs you will often see formats that start with `1X` or a string like `' '`. The reason is historical: when output went to a line printer, the first character of each line was not printed but used for *carriage control* (a blank meant "normal line", `'1'` meant "new page", `'0'` "skip a line"). Modern systems ignore this convention, but you need to know it when reading old code. In old code you may also meet *Hollerith* constants such as `5HHELLO` (the 5 characters after the H), which were used for text in formats before character strings existed; today simply write `'HELLO'`.

In this example each write statement used a different format statement. But it is perfectly fine to use the same format statement for many different write statements. In fact, this is one of the main advantages of using format statements. This feature is handy when you print tables for instance, and want each row to have the same format.

17.4 Format strings in read/write statements

Instead of specifying the format code in a separate format statement, one can give the format code in the read/write statement directly. For example, the statement

```
      write (*,'(A, F8.3)') 'The answer is x = ', x
```

is equivalent to

```
      write (*,990) 'The answer is x = ', x
 990 format (A, F8.3)
```

Sometimes text strings are given in the format statements, e.g. the following version is also equivalent:

```
      write (*,999) x
 999 format ('The answer is x = ', F8.3)
```

17.5 Implicit loops and repeat counts

Now let us do a more complicated example. Say you have a two-dimensional array of integers and want to print the upper left 5 by 10 submatrix with 10 values each on 5 rows. Here is how:

```
      do 10 i = 1, 5
         write(*,1000) (a(i,j), j=1,10)
   10 continue
 1000 format (10I6)
```

We have an explicit do loop over the rows and an *implicit* loop over the column index j. The repeat count 10 in `10I6` means ten integer fields, each 6 characters wide. It matters: with `format (I6)` the format describes only one number, and each write statement would print its ten values on ten separate lines (see the rule for "running out" of a format below).

Often a format statement involves repetition, for example

```
  950 format (2X, I3, 2X, I3, 2X, I3, 2X, I3)
```

There is a shorthand notation for this:

```
  950 format (4(2X, I3))
```

It is also possible to allow repetition without explicitly stating how many times the format should be repeated. Suppose you have a real vector `x` where you want to print the first 50 elements, with ten elements on each line. Here is one way:

```
      write(*,1010) (x(i), i=1,50)
 1010 format (10F7.2)
```

The format statement says ten numbers should be printed. But in the write statement we try to print 50 numbers. So after the ten first numbers have been printed, a new line is started and the same format is automatically used for the next ten numbers, and so on. This is called *format reversion*.

Be careful to match the format codes to the types of the values: `x` is real, so it needs an F (or E) code. Writing a real number with an I code is an error, and gfortran stops the program with a message like *Expected INTEGER for item 1 in formatted transfer, got REAL*.

Implicit do-loops can be multi-dimensional and can be used to make a read or write statement difficult to understand. You should avoid using implicit loops which are much more complicated than the ones shown here.

17.6 Internal files

The unit in a read or write statement does not have to be a file or the screen: it can also be a *character variable*. Such a variable is called an *internal file*. Writing to an internal file converts numbers into text, and reading from it converts text into numbers, using the same format codes as for ordinary I/O. This is the standard Fortran 77 way to convert between numbers and strings:

```
1       program dates
2       implicit none
3       character*8 text
4       integer year, month, day, date
5
6  c Text to numbers: read from the character variable text
7       text = '20230415'
8       read (text, '(I4,I2,I2)') year, month, day
9       write (*,*) 'Year:', year, '  Month:', month, '  Day:', day
10
11 c Numbers to text: write into the character variable text
12      date = 20240101
13      write (text, '(I8)') date
14      write (*,*) 'As text: [', text, ']'
15
16 c Build a date with leading zeros
17      year = 2025
18      month = 7
19      day = 4
20      write (text, '(I4,I2.2,I2.2)') year, month, day
21      write (*,*) 'Built:   [', text, ']'
22
23      stop
24      end
```

Output:

```
 Year:        2023   Month:           4   Day:          15
 As text: [20240101]
 Built:   [20250704]
```

Internal files are also useful to build file names, e.g. `write (fname, '(A,I3.3,A)') 'data', k, '.txt'` gives `data007.txt` when `k` is 7. Note

that Fortran 77 requires a format for internal files; list-directed I/O (*) on internal files was only allowed from Fortran 90 on.

17.7 Putting it together: a table of circle areas

We end this chapter by returning, once more, to our circle program. This version reads any number of radii from the file `radii.dat` (one per line), computes the areas with an external function, and prints a neat table:

```
      program circle
      implicit none
      integer u, n
      parameter (u = 10)
      real r, carea

      open (u, file='radii.dat', status='old', err=900)

      write (*,'(A)') '   Radius        Area'
      write (*,'(A)') '  -------  ----------'
      n = 0
   10 read (u, *, end=20) r
         n = n + 1
         write (*,'(F9.3, F12.4)') r, carea(r)
         goto 10
   20 continue
      close (u)
      write (*,'(I4, A)') n, ' circles'
      stop

  900 write (*,*) 'Cannot open the file radii.dat'
      stop
      end

      real function carea(r)
      implicit none
      real r, pi
      parameter (pi = 3.14159265)

      carea = pi * r * r

      return
      end
```

If `radii.dat` contains the numbers 0.5, 1, 2, 10 and 25.5, the output is:

```
   Radius        Area
  -------  ----------
    0.500      0.7854
    1.000      3.1416
    2.000     12.5664
   10.000    314.1593
   25.500   2042.8208
   5 circles
```

Compare this with the first version in Chapter 3: the program now uses a named constant, a function, a file, a while-loop built with `goto`, end-of-file and error handling, and formatted output. The headings were written with the same field widths (9 and 12 characters) as the numbers, so the columns line up.

17.8 NAMELIST I/O

In addition to formatted and list-directed I/O, there is a third method called NAMELIST I/O. Strictly speaking it is *not* part of Fortran 77: it was a common extension (many Fortran 77 compilers supported it, with small differences) and became standard in Fortran 90. All current compilers support it. This feature allows you to read and write groups of variables by name, which can be particularly convenient for managing program input parameters or debugging.

17.8.1 Defining a NAMELIST group

Before using NAMELIST I/O, you must define one or more NAMELIST groups in your program. Each group consists of a name and a list of variables:

```
      NAMELIST /group_name/ variable_list
```

For example:

```
      real x, y, z
      integer count, flag
      character*20 title

      NAMELIST /params/ x, y, z, count, flag
      NAMELIST /header/ title
```

This defines two NAMELIST groups: "params" containing five numeric variables, and "header" containing one character variable.

The NAMELIST statement should appear in the declaration section of your program, after all variable declarations but before any executable statements.

17.8.2 Reading with NAMELIST

To read values using NAMELIST, use the READ statement with the NAMELIST group name:

```
      READ ([UNIT=]u, NML=group_name [,IOSTAT=ios] [,ERR=s] [,END=s])
```

For example:

```
      READ (*, NML=params)
```

The input format for NAMELIST is special. It begins with an ampersand followed by the group name, contains variable assignments, and ends with a slash:

```
      &group_name variable=value [,variable=value]... /
```

For example, given the NAMELIST group "params" defined above, valid input would be:

```
      &params x=1.0, y=2.0, z=3.0, count=10, flag=1 /
```

The input can span multiple lines, and you only need to provide values for the variables you want to change. Variables not mentioned in the input retain their current values. Also, the order of variables in the input does not have to match the order in the NAMELIST declaration.

Array input with NAMELIST

NAMELIST makes it easy to input array values. You can input an entire array, a section of an array, or individual elements:

```
      real a(10), matrix(5,5)
      NAMELIST /arrays/ a, matrix

      READ (*, NML=arrays)
```

Valid input might look like:

```
      &arrays
        a = 10*0.0,
        matrix(1,1) = 1.0,
        matrix(2,2) = 2.0,
        matrix(3,3) = 3.0,
        matrix(4,4) = 4.0,
        matrix(5,5) = 5.0
      /
```

Or using array section notation (which varies by compiler implementation):

```
      &arrays
        a = 10*0.0,
        matrix(1:5:1,1:5:1) = 25*0.0,
        matrix(1,1) = 1.0,
        matrix(2,2) = 2.0,
        matrix(3,3) = 3.0,
        matrix(4,4) = 4.0,
        matrix(5,5) = 5.0
      /
```

17.8.3 Writing with NAMELIST

Similarly, you can write values using NAMELIST:

```
      WRITE ([UNIT=]u, NML=group_name [,IOSTAT=ios] [,ERR=s])
```

For example:

```
      WRITE (*, NML=params)
```

This would produce output in the same format as NAMELIST input:

```
      &PARAMS
       X=   1.000000    ,
       Y=   2.000000    ,
       Z=   3.000000    ,
       COUNT=         10,
       FLAG=          1
      /
```

The exact format of the output will depend on the compiler, but it will always be in a form that can be read back with a NAMELIST READ statement.

17.8.4 Advantages of NAMELIST I/O

NAMELIST I/O offers several advantages over other I/O methods:

1. **Self-documenting input** - The variables are referenced by name in the input file.
2. **Flexibility** - You can read only the variables you need to change.
3. **Convenience** - No need to write FORMAT statements or match input order.
4. **Debugging** - Easy to dump variables and their values during program execution.

17.8.5 Example: Parameter input with NAMELIST

Here's a complete example showing how NAMELIST can be used for parameter input:

```
      program simulation
      implicit none
      real dt, tend, x0, v0
      integer nsteps, output_freq
      character*30 title, output_file
      logical debug

c Declare namelist groups
      NAMELIST /params/ dt, tend, nsteps, output_freq, debug
```

```
      NAMELIST /initial/ x0, v0
      NAMELIST /output/ title, output_file

c Set defaults
      dt = 0.01
      tend = 10.0
      nsteps = 1000
      output_freq = 10
      debug = .FALSE.

      x0 = 0.0
      v0 = 0.0

      title = 'Simulation Results'
      output_file = 'output.dat'

c Read parameters from standard input
      write(*,*) 'Enter simulation parameters:'
      read(*, NML=params, END=100)
      read(*, NML=initial, END=100)
      read(*, NML=output, END=100)

c Echo parameters back to the user
      write(*,*) 'Running simulation with parameters:'
      write(*, NML=params)
      write(*, NML=initial)
      write(*, NML=output)

c Run simulation (not shown here)

  100 continue
      stop
      end
```

With this program, a user could provide input as simple as:

```
      &params dt=0.005, tend=20.0 /
      &initial x0=1.0, v0=-0.5 /
      &output title='Fall 2023 Simulation Run' /
```

And the program would use those values along with defaults for the unspecified parameters. With gfortran, the echoed values look like this:

```
&PARAMS
 DT=  4.99999989E-03,
 TEND=  20.0000000     ,
 NSTEPS=1000        ,
 OUTPUT_FREQ=10           ,
 DEBUG=F,
 /
```

Notice that `dt` is printed as `4.99999989E-03` although we typed 0.005: as we saw in Chapter 5, most decimal fractions cannot be stored exactly in a `real`.

17.8.6 Limitations of NAMELIST I/O

While NAMELIST is a powerful feature, it has some limitations:

1. It's not part of the Fortran 77 standard (it is standard since Fortran 90, and widely implemented as an extension in Fortran 77 compilers).
2. Syntax details may vary slightly between compilers.
3. Derived types (in Fortran 90+) can't be included in NAMELIST groups in older compilers.
4. Some compilers restrict the types of expressions that can appear in NAMELIST input.

Despite these limitations, NAMELIST I/O is a valuable tool for many Fortran applications, especially those requiring flexible parameter input or extensive debugging output.

17.9 Exercises

Solutions are in the directory `code/exercises/ch17` that accompanies this tutorial.

1. Write a program that prints the multiplication table of the numbers 1 to 9, with a heading row and a separating line, using format statements with repeat counts and implied do-loops.
2. Using internal files to build the names, write a program that creates the files `out001.txt`, ..., `out005.txt`, each containing one line `This is file number k`. Then the program should open each file again, print its line, and obtain the number k back from the text with another internal read. Delete the files when closing them.
3. Without a computer, write down exactly what each of these statements prints (the brackets show where each field begins and ends). Then check your answers.

```
1      write (*,'(A,I5,A)')    '[', 12345678, ']'
2      write (*,'(A,F6.2,A)')  '[', 3.14159, ']'
3      write (*,'(A,E10.3,A)') '[', -0.000123, ']'
4      write (*,'(A,I3.3,A)')  '[', 7, ']'
5      write (*,'(A,A3,A)')    '[', 'Fortran', ']'
6      write (*,'(A,A10,A)')   '[', 'Fortran', ']'
7      write (*,'(A,L3,A)')    '[', .TRUE., ']'
8      write (*,'(A,F4.1,A)')  '[', -9.96, ']'
```

4. Write a program that prints the squares of $1, 2, \ldots, 25$, eight on each line, with a single write statement and the format `(8I6)`. How many lines are printed, and why?

5. Each line of the input has a date written as *yyyymmdd* and a time *hhmm*, separated by one blank, e.g. `20240315 1250`. Write a program that reads the parts with a format, by columns, and prints them as `2024-03-15 12:50`.

6. Write a program that reads a number of decimals d (from 0 to 7) and prints π with d decimals. Build the format at run time: write it into a character variable with an internal write, e.g. `'(F10.5)'` for $d = 5$, and then use the variable as the format.

7. Write a program that reads the parameters `nx`, `ny`, `xmin` and `xmax` of a grid with NAMELIST, with the defaults 11, 11, 0.0 and 1.0, and prints them back (also with NAMELIST) together with the spacing $dx = (x_{max} - x_{min})/(n_x - 1)$. Try it with the input `&grdpar nx=21, xmax=2.0 /`.

8. Extend the program for the table of circle areas of this chapter with a column for the circumference, and print the total area of all the circles at the end of the table.

Chapter 18

Numerical Software

Most computational problems in engineering can be broken down into well-known types of calculations, e.g., solving linear systems of equations, computing fast Fourier transforms, etc. Furthermore, software to solve these subtasks is often already available. Consequently, you only have to write a short *driver* routine for your particular problem. This way people don't have to reinvent the wheel over and over again.

For linear algebra and many other basic numerical computations there is high-quality free software available, much of it through Netlib. For some specialized problems, the best software must be purchased from a commercial company.

18.1 Netlib

Netlib (the NET LIBrary) is a large collection of freely available software, documents, and databases of interest to the numerical, scientific computing, and other communities. It was started in the 1980's at AT&T Bell Laboratories and Argonne National Laboratory, and is now maintained mainly at the University of Tennessee and Oak Ridge National Laboratory. The address is

https://www.netlib.org/

Netlib contains high-quality software that has been extensively tested, but (as all free software) it comes with no warranty and little (if any) support. Most of it is written in Fortran 77. In order to use the software, you download the source code and compile it yourself. A very useful feature is that for many routines Netlib can deliver the routine *together with all the routines it depends on*; we will use this in the section on LAPACK.

Two of the most popular packages at Netlib are the BLAS and LAPACK libraries, which we will describe in later sections. Other well-known Netlib

packages, all written in Fortran 77, are QUADPACK (numerical integration), FFTPACK (fast Fourier transforms), ODEPACK (ordinary differential equations) and MINPACK (nonlinear equations and least squares).

18.2 Other sources of numerical software

In this section we briefly mention some other important sources of software for numerical computations.

Optimized BLAS and LAPACK The reference BLAS and LAPACK from Netlib are written for clarity and portability, not for speed. Optimized versions with exactly the same calling sequences are available for most computers, for example the free OpenBLAS, Intel's oneMKL, and Apple's Accelerate framework. A program that calls BLAS or LAPACK can use any of them without changes: only the library you link with changes.

NAG The Numerical Algorithms Group sells a large Fortran library with well over 1000 user-callable routines for solving general applied math problems, including: ordinary and partial differential equations, optimization problems, FFT and other transforms, quadrature, linear algebra, non-linear equations, integral equations, statistics, and more.

IMSL The IMSL Fortran Numerical Library is another commercial library that covers most of the areas of the NAG library, with special strength in statistics.

Numerical Recipes The books *Numerical Recipes* by Press, Teukolsky, Vetterling and Flannery (with editions in Fortran 77, Fortran 90, C and C++) are very popular among engineers because they can be used as cookbooks where you can find a "recipe" to solve the problem at hand, together with a clear explanation of the method. They are excellent for learning. Note, however, that the code is copyrighted and its license is restrictive, and that for serious work the libraries mentioned above are usually more robust. See `https://numerical.recipes`.

Most of the software listed above can also be called from C and other languages.

18.3 BLAS

BLAS is an acronym for Basic Linear Algebra Subroutines. As the name indicates, it contains subprograms for basic operations on vectors and matrices. BLAS was designed to be used as a building block in other codes, for example LAPACK. The source code for BLAS is available through Netlib. However, many computer vendors will have a special version of BLAS tuned for maximal speed and efficiency on their computer. This is one of the main advantages of BLAS: the calling sequences are standardized so that programs that call BLAS will work on any computer that has BLAS installed. If you have a fast version of BLAS, you will also get high performance on all programs that call BLAS. Hence BLAS provides a simple and portable way to achieve high performance for calculations involving linear algebra. LAPACK is a higher-level package built on the same ideas.

18.3.1 Levels and naming conventions

The BLAS subroutines can be divided into three *levels*:

- **Level 1:** Vector-vector operations. $O(n)$ data and $O(n)$ work.
- **Level 2:** Matrix-vector operations. $O(n^2)$ data and $O(n^2)$ work.
- **Level 3:** Matrix-matrix operations. $O(n^2)$ data and $O(n^3)$ work.

Each BLAS and LAPACK routine comes in several versions, one for each precision (data type). The first letter of the subprogram name indicates the precision used:

```
1       S       Real single precision.
2       D       Real double precision.
3       C       Complex single precision.
4       Z       Complex double precision.
```

Complex double precision is not strictly defined in Fortran 77, but most compilers will accept one of the following declarations:

```
1       double complex list-of-variables
2       complex*16     list-of-variables
```

18.3.2 BLAS 1

Some of the BLAS 1 subprograms are:

- xCOPY - copy one vector to another
- xSWAP - swap two vectors
- xSCAL - scale a vector by a constant

- xAXPY - add a multiple of one vector to another
- xDOT - inner product
- xASUM - 1-norm of a vector
- xNRM2 - 2-norm of a vector
- IxAMAX - find maximal entry in a vector

The first letter (x) can be any of the letters S,D,C,Z depending on the precision. A quick reference to BLAS 1 can be found at http://www.netlib.org/blas/blasqr.ps.

18.3.3 BLAS 2

Some of the BLAS 2 subprograms are:

- xGEMV - general matrix-vector multiplication
- xGER - general rank-1 update
- xSYR2 - symmetric rank-2 update
- xTRSV - solve a triangular system of equations

A detailed description of BLAS 2 can be found at http://www.netlib.org/blas/blas2-paper.ps.

18.3.4 BLAS 3

Some of the BLAS 3 subprograms are:

- xGEMM - general matrix-matrix multiplication
- xSYMM - symmetric matrix-matrix multiplication
- xSYRK - symmetric rank-k update
- xSYR2K - symmetric rank-2k update

The more advanced matrix operations, like solving a linear system of equations, are contained in LAPACK. A detailed description of BLAS 3 can be found at http://www.netlib.org/blas/blas3-paper.ps.

18.3.5 Examples

Let us first look at a very simple BLAS routine, SSCAL. The call sequence is

```
1       call SSCAL ( n, a, x, incx )
```

Here x is the vector, n is the length (number of elements in x we wish to use), and a is the scalar by which we want to multiply x. The last argument *incx* is the *increment*. Usually, *incx=1* and the vector x corresponds directly to the one-dimensional Fortran array x. For *incx>1* it specifies how many elements in the array we should "jump" between each element of the vector x. For example, if *incx=2* it means we should only scale every other element (note: the physical dimension of the array x should then be at least *2n-1*). Consider these examples where x has been declared as `real x(100)`.

```
1       call SSCAL(100, a, x, 1)
2       call SSCAL( 50, a, x(51), 1)
3       call SSCAL( 50, a, x(2), 2)
```

The first line will scale all 100 elements of x by a. The next line will only scale the last 50 elements of x, i.e. x(51) to x(100). (A common slip is to write `x(50)`: that scales x(50) to x(99) and leaves x(100) untouched.) The last line will scale all the even indices of x by a: x(2), x(4), ..., x(100).

Observe that the array x will be overwritten by the new values. If you need to preserve a copy of the old x, you have to make a copy first, e.g., by using SCOPY.

Now consider a more complicated example. Suppose you have two 2-dimensional arrays A and B, and you are asked to find the *(i,j)* entry of the product A*B. This is easily done by computing the inner product of row i from A and column j of B. We can use the BLAS 1 routine SDOT. Unlike SSCAL, SDOT is a *function*: it returns the inner product as its value, so it must be declared (`real sdot`) and used in an expression. The only difficulty is to figure out the correct indices and increments. The calling sequence for SDOT is

```
1       s = SDOT ( n, x, incx, y, incy )
```

Suppose the array declarations were

```
1       real A(lda,lda)
2       real B(ldb,ldb)
```

but in the program you know that the actual size of A is *m*p* and for B it is *p*n*. The *i*'th row of A starts at the element *A(i,1)*. But since Fortran stores 2-dimensional arrays down columns, the next row element *A(i,2)* will be stored *lda* elements later in memory (since *lda* is the length of a column). Hence we set *incx = lda*. For the column in B there is no such problem, the elements are stored consecutively so *incy = 1*. The length of the inner product calculation is p. Hence the answer is

```
1       cij = SDOT ( p, A(i,1), lda, B(1,j), 1 )
```

18.3.6 How to get the BLAS

First of all you should check if you already have BLAS on your system. On Linux it is usually one package away, e.g. `sudo apt install libblas-dev` (Debian/Ubuntu) or `sudo dnf install blas-devel` (Fedora). If not, you can get the *reference* BLAS from Netlib at https://www.netlib.org/blas/ and build it yourself with gfortran. The whole library comes as a single archive:

```
curl -O https://www.netlib.org/blas/blas.tgz
tar xzf blas.tgz
cd BLAS-3.12.0
gfortran -O2 -c *.f *.f90
ar cr libblas.a *.o
ranlib libblas.a
```

A few remarks:

- The version number in the directory name (here 3.12.0) changes with each release; use whatever `tar` created.
- Some `tar` programs print warnings like *Ignoring unknown extended header keyword*. They are harmless: the archive contains macOS metadata.
- Recent versions include a few `.f90` files (e.g. `snrm2.f90`), so you need a compiler that also understands Fortran 90, such as gfortran.
- Instead of the last three commands you can run `make`, which uses the settings in `make.inc` and produces a library named `blas_LINUX.a`.
- If you only need one or two routines, you can download them individually, e.g. https://www.netlib.org/blas/sscal.f, and compile them together with your program.

To link your program with the library (here assumed to be in the current directory):

```
gfortran myprog.f -L. -lblas -o myprog
```

The reference BLAS is meant to be correct and portable, not fast. For serious computations use an optimized implementation with the same interface, such as OpenBLAS or the vendor libraries (Intel MKL, Apple Accelerate); your program does not need to change, only the library you link with.

18.3.7 Documentation

The BLAS routines are almost self-explanatory. Once you know which routine you need, fetch it and read the header section that explains the input and output parameters in detail. We will look at an example in the next section when we address the LAPACK routines.

18.4 LAPACK

LAPACK is a collection of Fortran subprograms for advanced linear algebra problems like solving systems of simultaneous linear equations, least-squares solutions of linear systems of equations, eigenvalue problems, and singular value problems. LAPACK replaces the older packages LINPACK and EISPACK. LAPACK subroutines were written to exploit BLAS as much as possible.

18.4.1 Routines

Probably the most widely used LAPACK routines are the ones that solve systems of linear equations:

- xGESV - Solve AX=B for a general matrix A
- xPOSV - Solve AX=B for a symmetric positive definite matrix A
- xGBSV - Solve AX=B for a general banded matrix A

There are many more routines for other special types of matrices.

The source code is available from Netlib at https://www.netlib.org/lapack/. The complete LAPACK User's Guide is also on the Web.

18.4.2 Documentation

Just like the BLAS routines, the LAPACK routines are virtually self-explanatory. Details of the input and output parameters for any given subroutine are contained in the header section of the file. For example, here is the header section of the LAPACK subroutine SGESV:

```
1       SUBROUTINE SGESV( N, NRHS, A, LDA, IPIV, B, LDB, INFO )
2  *
3  *  -- LAPACK driver routine (version 2.0) --
4  *     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
5  *     Courant Institute, Argonne National Lab, and Rice University
6  *     March 31, 1993
7  *
8  *     .. Scalar Arguments ..
9       INTEGER            INFO, LDA, LDB, N, NRHS
10 *     ..
11 *     .. Array Arguments ..
12      INTEGER            IPIV( * )
13      REAL               A( LDA, * ), B( LDB, * )
14 *     ..
15 *
16 *  Purpose
17 *  =======
18 *
19 *  SGESV computes the solution to a real system of linear equations
```

```
*     A * X = B,
*  where A is an N-by-N matrix and X and B are N-by-NRHS matrices.
*
*  The LU decomposition with partial pivoting and row interchanges is
*  used to factor A as
*     A = P * L * U,
*  where P is a permutation matrix, L is unit lower triangular, and U is
*  upper triangular.  The factored form of A is then used to solve the
*  system of equations A * X = B.
*
*  Arguments
*  =========
*
*  N       (input) INTEGER
*          The number of linear equations, i.e., the order of the
*          matrix A.  N >= 0.
*
*  NRHS    (input) INTEGER
*          The number of right hand sides, i.e., the number of columns
*          of the matrix B.  NRHS >= 0.
*
*  A       (input/output) REAL array, dimension (LDA,N)
*          On entry, the N-by-N coefficient matrix A.
*          On exit, the factors L and U from the factorization
*          A = P*L*U; the unit diagonal elements of L are not stored.
*
*  LDA     (input) INTEGER
*          The leading dimension of the array A.  LDA >= max(1,N).
*
*  IPIV    (output) INTEGER array, dimension (N)
*          The pivot indices that define the permutation matrix P;
*          row i of the matrix was interchanged with row IPIV(i).
*
*  B       (input/output) REAL array, dimension (LDB,NRHS)
*          On entry, the N-by-NRHS matrix of right hand side matrix B.
*          On exit, if INFO = 0, the N-by-NRHS solution matrix X.
*
*  LDB     (input) INTEGER
*          The leading dimension of the array B.  LDB >= max(1,N).
*
*  INFO    (output) INTEGER
*          = 0:  successful exit
*          < 0: if INFO = -i, the i-th argument had an illegal value
*          > 0:  if INFO = i, U(i,i) is exactly zero.  The factorization
*                has been completed, but the factor U is exactly
*                singular, so the solution could not be computed.
*
*  =====================================================================
```

The header tells us everything we need to call the routine. In particular, `A` and `B` are *overwritten*: on return, `A` contains the LU factors and `B` the solution. If you still need the original matrix, make a copy before the call.

18.4.3 Example: solving a linear system

The following program solves the system

$$\begin{array}{rcrcrcr} 2x & + & y & - & z & = & 8 \\ -3x & - & y & + & 2z & = & -11 \\ -2x & + & y & + & 2z & = & -3 \end{array}$$

with `DGESV`, the double precision version of the routine documented above:

```
1       program solve
2       implicit none
3       integer n, nrhs, lda, ldb
4       parameter (n=3, nrhs=1, lda=n, ldb=n)
5       double precision A(lda,n), b(ldb)
6       integer ipiv(n), info, i
7
8  c The system   2x +  y -  z =   8
9  c              -3x -  y + 2z = -11
10 c              -2x +  y + 2z =  -3
11 c A is given column by column (that is how Fortran stores it)
12      data A / 2.0d0, -3.0d0, -2.0d0,
13     +         1.0d0, -1.0d0,  1.0d0,
14     +        -1.0d0,  2.0d0,  2.0d0 /
15      data b / 8.0d0, -11.0d0, -3.0d0 /
16
17      call dgesv(n, nrhs, A, lda, ipiv, b, ldb, info)
18
19      if (info .ne. 0) then
20         write (*,*) 'DGESV failed, info =', info
21         stop
22      endif
23      write (*,'(A,I1,A,F10.6)') ('x(', i, ') = ', b(i), i=1,n)
24
25      stop
26      end
```

Output:

```
x(1) =   2.000000
x(2) =   3.000000
x(3) =  -1.000000
```

Some things to notice:

- Always check `info` after calling a LAPACK routine. A positive value from `DGESV` means that the matrix is singular and there is no (unique) solution.
- The matrix is initialized with a `data` statement, which fills it column by column. Writing the numbers row by row is a classic mistake that silently solves the system with the transposed matrix.

- `b` is declared as a one-dimensional array although the routine expects `B(LDB,*)`. With one right-hand side this is allowed, because a column of a two-dimensional array is stored exactly like a vector (see Chapter 12).

18.4.4 How to get LAPACK

On Linux, LAPACK is usually available as a package, e.g. `sudo apt install liblapack-dev` (Debian/Ubuntu) or `sudo dnf install lapack-devel` (Fedora). Then the program above is compiled with

```
gfortran solve.f -llapack -lblas -o solve
```

(LAPACK calls BLAS, so `-lblas` must come after `-llapack`.)

The whole of LAPACK is large, but if you only need a few routines, Netlib can give you a routine *together with all the routines it depends on*, including the BLAS routines it uses. For `DGESV`:

```
curl -o dgesv.tgz "https://www.netlib.org/cgi-bin/netlibfiles.tgz?
    format=tgz&filename=lapack/double/dgesv.f"
tar xzf dgesv.tgz
gfortran -O2 -c lapack/*/*.f
ar cr liblapack.a *.o
ranlib liblapack.a
gfortran solve.f -L. -llapack -o solve
```

Here the library contains everything `DGESV` needs (17 files), so no separate BLAS library is required. The routines delivered this way may come from an older LAPACK version than the current one, which is fine for learning. For the complete, current LAPACK, download https://www.netlib.org/lapack/ or use the official repository https://github.com/Reference-LAPACK/lapack.

18.5 Using libraries

A Fortran package of subprograms may contain hundreds of files. It is very slow and inconvenient to recompile these files every time you want to use any of the subroutines. You can avoid this by making a *library file*. A library is a collection of compiled object files, so you only have to compile your own main (driver) program and then link it with the library. (Linking is much faster than compiling.)

Static libraries have file names starting with `lib` and ending in `.a`. Libraries installed on the system (for example with the package manager of a Linux distribution) are usually in directories such as `/usr/lib` or `/usr/local/lib`, where the compiler finds them automatically. For example, if the BLAS library is installed as `libblas.a` (or as a shared library `libblas.so`), you use the `-l` option to link it together with your main program, e.g.

```
gfortran main.f -lblas -o main
```

Note that you write `-lblas`, not `-llibblas.a`: the compiler adds the prefix `lib` and the extension itself. You can link several files with several libraries at the same time if you wish:

```
gfortran main.f mysub.f -llapack -lblas -o main
```

The order you list the files and libraries is significant: the linker reads them from left to right, and takes from each library only the routines that are needed by the files read *before* it. Therefore libraries must come after the programs that use them, and in the example above `-llapack` should be listed before `-lblas`, since LAPACK calls BLAS routines.

18.5.1 Creating your own library

If you want to create your own library, you compile the source code to object code and then collect all the object files into one library file with the program `ar`. As an example, let us put the subroutines `saxpy`, `matvec`, `scale` and `mscale` of Chapter 12, each in its own file, into a library called `libmyla.a`:

```
gfortran -c saxpy.f matvec.f scale.f mscale.f
ar rcs libmyla.a saxpy.o matvec.o scale.o mscale.o
```

The options `rcs` of `ar` mean: *r*eplace (or add) the files in the library, *c*reate the library if it does not exist, and write an index of the routines (on older systems the index was made with a separate command, `ranlib`). The command `ar t libmyla.a` lists the contents of a library. If you have the library file in the current directory, you can link with it as follows; the option `-L.` tells the linker to look for libraries also in the current directory ("`.`"):

```
gfortran sclmat.f -L. -lmyla -o sclmat
```

Here `sclmat.f` is the test program of Chapter 12. If you put `-L. -lmyla` *before* `sclmat.f`, the linker reports errors such as *undefined reference to 'mscale_'*: when it read the library, no routine from it was needed yet.

(The trailing underscore in `mscale_` is added by gfortran to the names of all external routines; you will see it in the error messages of the linker, and you need to know about it if you ever call Fortran from C.)

One advantage of libraries is that you only compile them once but you can use them many times. The commands above work in the same way on Linux, macOS and Windows (with MinGW or MSYS2).

18.6 Searching for mathematical software

A common situation is that you need a piece of code to solve a particular problem. You believe somebody must have written software to solve this type of problem already, but you do not know how to find it. Here are some suggestions:

1. Ask your co-workers and colleagues. Always do this first, otherwise you can waste lots of time trying to find something the person next-door may have already.

2. Check Netlib. The list of libraries is a good starting point, and there is also a search feature.

3. Look it up in GAMS, the Guide to Available Mathematical Software from NIST, which classifies numerical software by the type of problem it solves.

4. Look at the list of Fortran packages maintained by the Fortran community at `fortran-lang.org`, or search a code hosting site such as GitHub.

5. Ask in the Fortran Discourse forum, where many experienced Fortran programmers answer questions.

Whatever you find, check its license before you use it in your own programs, and test it on a problem whose solution you know.

18.7 Exercises

Solutions are in the directory `code/exercises/ch18` that accompanies this tutorial. To run them you need the BLAS and LAPACK libraries (see `get_blas.sh` and `get_lapack.sh` in the `code` directory).

1. Let $x = (1, 2, \ldots, 6)$ and $y = (10, 10, \ldots, 10)$. Use the BLAS routine `SAXPY(n, a, x, incx, y, incy)` to compute $y := 2x + y$, first for all the elements, and then (starting again from the original y) only for the elements with odd index, using increments of 2. Print y in both cases.

2. Use the BLAS functions `SNRM2` and `SDOT` to compute the lengths of the vectors $u = (1, 0, 1)$ and $v = (0, 1, 1)$ and the angle between them, $\cos\theta = u \cdot v / (|u|\,|v|)$. Remember that functions must be declared with their type.

3. The BLAS function `ISAMAX(n, x, incx)` returns the *position* of the element of x with the largest absolute value. Use it to find and print that element of $x = (3, -8.5, 2, 7, -1, 8, 0.5)$.

4. Let A be the 2×3 matrix with $a_{ik} = i + k$ and B the 3×2 matrix with $b_{kj} = kj$, both stored in 5×5 arrays. Compute $C = AB$ entry by entry with `SDOT`, as in the example of this chapter, and then with a single call of the BLAS 3 routine `SGEMM('N', 'N', m, n, p, 1.0, A, lda, B, ldb, 0.0, C, ldc)`, which computes $C := 1.0\,AB + 0.0\,C$. Compare the results.

5. Modify the program `solve.f` of this chapter to solve, with a single call of `DGESV`, the system with two right-hand sides: the one of the example and $(2, -2, 1)$. (Hint: `B` becomes a 3×2 array, and `NRHS` is 2.)

6. Call `DGESV` with the singular matrices whose rows are $(1, 2, 3), (2, 4, 6), (1, 1, 1)$ and $(1, 2, 3), (4, 5, 6), (5, 7, 9)$, and the right-hand side $(1, 2, 3)$. What is the value of `INFO` in each case? Why does `DGESV` not detect that the second matrix is singular?

7. Put the subroutines `saxpy`, `matvec`, `scale` and `mscale` of Chapter 12 into a library `libmyla.a`, list its contents with `ar t`, and link the program `sclmat.f` with it. What happens if you put `-L. -lmyla` before `sclmat.f` on the command line?

8. In the program `solve.f`, write the matrix in the data statement row by row, as it appears in the system of equations. What solution does the program print? Which system has it solved?

Chapter 19

Fortran programming style

There are many different programming styles, but here are some general guidelines that are fairly non-controversial.

19.1 Portability

To ensure portability, use only standard Fortran 77, plus a small number of extensions that are supported by every compiler in use today. The extensions used in this tutorial are:

- lower case letters;
- `implicit none`;
- names longer than six characters and names containing underscores;
- comments starting with `!` at the end of a line;
- the `include` statement, `do while` and `enddo`, `NAMELIST` and the `POSITION` specifier, which we pointed out where they were used.

All of them are standard in Fortran 90 and later. Avoid other extensions, such as `real*8` or compiler-specific functions, unless you really need them. Compiling your program with more than one compiler, or with `-std=f95 -Wall` with gfortran, is a good way to discover unintended non-standard code.

19.2 Program structure

The overall program structure should be modular. Each subprogram should solve a well-defined task. Many people prefer to write each subprogram in a separate file.

19.3 Comments

Write legible code, but also add comments in the source explaining what is happening! It is especially important to have a good header for each subprogram that explains each input/output argument and what the subprogram does. The header of the LAPACK routine `SGESV` in Chapter 18 is an excellent model.

19.4 Indentation

Always use proper indentation for loops and if blocks as demonstrated in this tutorial.

19.5 Variables

Always declare all variables, and use `implicit none` to make the compiler check that you did (Chapter 11 shows what can happen otherwise). If you must write strictly standard Fortran 77, stick to a maximum of 6 characters for names, or at the very least make sure the first 6 characters are unique.

Choose `double precision` for floating point computations unless you have a good reason to use `real`, and never test floating point numbers for exact equality (Chapter 5).

19.6 Subprograms

Never let functions have "side effects", i.e. do not change the value of the input parameters. Use subroutines in such cases.

In the declarations, separate parameters, common blocks, and local variables.

Minimize the use of common blocks. If a local variable must keep its value between calls, declare it with `save`.

Never pass a constant or an expression as an argument that the subprogram may change (Chapter 11).

19.7 Goto

Minimize the use of *goto*. Unfortunately it is necessary to use *goto* in some loops since *while* is not standard Fortran 77; when you do, use the patterns of Chapter 9, so that the structure of the loop is clear. Avoid the arithmetic `if`, the assigned `goto` and alternate returns in new code.

19.8 Arrays

In many cases it is best to declare all large arrays in the main program and then pass them as arguments to the various subroutines. This way all the space allocation is done in one place. Remember to pass the leading dimensions. Avoid unnecessary "reshaping" of matrices.

Check array sizes before you use them, as the program of Section 15.11 does, and use the compiler option `-fcheck=bounds` while you develop your program.

19.9 Efficiency concerns

When you have a double loop accessing a two-dimensional array, it is usually best to have the first (row) index in the innermost loop. This is because of the storage scheme in Fortran: consecutive values of the first index are next to each other in memory, and modern processors are much faster when they access memory in order.

```
c Good: the inner loop runs over the first index
      do 20 j = 1, n
         do 10 i = 1, m
            A(i,j) = 0.0
   10    continue
   20 continue
```

When you have if-then-elseif statements with multiple conditions, try to place the most likely conditions first.

Do not spend time optimizing code before you know where the program spends its time. And for standard tasks, such as linear algebra, use well-tested libraries (Chapter 18): they are usually both faster and more reliable than code you write yourself.

19.10 Exercises

Solutions are in the directory `code/exercises/ch19` that accompanies this tutorial.

1. The program `ugly.f` in that directory reads numbers until the end of the input, ignores those that are not positive, and prints the numbers that are larger than the average, and the average. List everything that is wrong with its style, and rewrite it following the guidelines of this chapter. Does the original program have any real bugs?

```
      program ugly
      dimension x(100)
      n=0
      s=0.
```

```
      1 read(*,*,end=2) v
        if(v)1,1,3
      3 n=n+1
        x(n)=v
        s=s+v
        goto 1
      2 a=s/n
        do 5 i=1,n
      5 if(x(i).gt.a) write(*,*) x(i)
        write(*,*) a
        end
```

2. Write a program that sums all the elements of a 2000×2000 double precision matrix twice: once with the first index in the inner loop, and once with the second index in the inner loop. Measure the time of each version with the Fortran 95 intrinsic subroutine `cpu_time(t)`, which every current compiler supports. Which version is faster, and why?

3. Compile the program `ext.f` in that directory with `gfortran -std=f95 -Wall`. It uses `real*8`, `do while`, a comment with `!` and a name with an underscore. Which of these extensions are reported, and why not the others? Rewrite the program in standard Fortran 77.

4. Write a header in the style of the LAPACK routine `SGESV` (Chapter 18) for the subroutine `mscale` of Chapter 12, explaining its purpose and each of its arguments.

Chapter 20

Debugging hints

A large part of the time it takes to develop software is spent on debugging and testing. This shows how important it is to write good code in the first place: declare all variables, use `implicit none`, keep subprograms small, and test each of them as soon as it is written.

Still, we all discover *bugs* from time to time. Here are some hints for how to track them down.

20.1 Useful compiler options

Most Fortran compilers have a set of options that make the compiler check more things, either when it compiles the program or while the program runs. The most useful ones for gfortran are:

`-Wall` Turns on the most important warnings: unused and uninitialized variables, lines longer than 72 columns, implicit conversions that lose information, and more. Read the warnings!

`-fimplicit-none` Has the same effect as `implicit none` in every program unit.

`-fcheck=all` Turns on all run-time checks, in particular array bounds checking (`-fcheck=bounds`). The program becomes slower, so use it while developing and testing.

`-g` Includes debugging information, so that error messages and debuggers can refer to line numbers of your source code.

`-fbacktrace` When the program stops with a run-time error, prints the sequence of calls that led to the error (on by default in recent versions of gfortran).

-ffpe-trap=invalid,zero,overflow Stops the program at the first invalid floating point operation (such as the square root of a negative number), division by zero, or overflow, instead of silently continuing with the special values `NaN` ("not a number") or `Infinity`.

-finit-real=snan Initializes all real variables with a special "signaling NaN". Together with `-ffpe-trap=invalid`, this makes the program stop when it uses a real variable that was never given a value.

-std=f95 Reports features that are not standard Fortran 95, which includes all the non-standard extensions to Fortran 77 and the features that were deleted from the language.

A good combination while developing a program is

```
gfortran -g -Wall -fcheck=all -ffpe-trap=invalid,zero,overflow prog.
    f -o prog
```

As an example of what the checks can do, consider this program, which writes one element beyond the end of an array:

```
1       program bounds
2       implicit none
3       real x(10)
4       integer i
5
6       do 10 i = 1, 11
7          x(i) = real(i)
8    10 continue
9       write(*,*) x(1)
10
11      stop
12      end
```

Compiled without options, the program may crash with a message that does not say much (such as *Segmentation fault*), or, worse, it may run to the end and silently overwrite some other variable. With `-g -fcheck=bounds` the problem is reported exactly:

```
At line 7 of file bounds.f
Fortran runtime error: Index '11' of dimension 1 of array 'x' above
    upper bound of 10
```

In this simple case, a recent gfortran even notices the problem while compiling, if you use `-Wall`: *Warning: Array reference at (1) out of bounds (11 > 10) in loop.* In real programs, where the bounds are only known at run time, the run-time check is what saves you.

Similarly, a program that divides by zero or takes the square root of a negative number normally continues and prints `Infinity` or `NaN`, which may then spread through all later results. Compiled with `-g -ffpe-trap=invalid,zero,overflow`, it stops at the faulty operation:

```
Program received signal SIGFPE: Floating-point exception - erroneous
     arithmetic operation.
```

followed by a backtrace that shows where it happened.

20.2 Some common errors

Here are some common errors to watch out for:

- Make sure your lines end at column 72. The rest is ignored, without any error message unless you compile with `-Wall`! For example, if a statement `total = a + b + 25` is written so that the `2` of `25` is in column 72, the `5` in column 73 is simply dropped: the program computes `a + b + 2`, and gfortran only warns about it (*Line truncated*) with `-Wall`.
- Do your argument lists in the calling and the called program match, in number, order and type? The compiler cannot check this when the subprograms are in different files.
- Have you declared the type of every function you call (Chapter 11)?
- Have you passed a constant to a subprogram that changes its argument (Chapter 11)?
- Do your common blocks match?
- Have you done integer division when you wanted real division?
- Are your array indices within bounds? Do you pass the correct leading dimension?
- Does a variable keep its value between calls only because you forgot `save`?
- Do you compare floating point numbers for exact equality?
- Have you typed an *o* when you meant 0, an *l* when you meant 1?

20.3 Debugging tools

If you have a bug, you have to try to locate it. Syntax errors are easy to find: the compiler tells you where they are. The problem is when you have run-time errors. The old-fashioned way to find errors is to add *write* statements in your code and try to track the values of your variables. This still works, and is often the quickest method.

A more powerful tool is a *debugger*. With a debugger you can step through a program line by line or define your own break points, you can display the values of the variables you want to monitor, and much more. The standard debugger for gfortran is *gdb*, the GNU debugger. Compile your program with `-g` (and preferably without optimization) and start it with `gdb ./prog`; Chapter 4 lists the most important commands. Most integrated development environments, such as Visual Studio Code with a Fortran extension, offer a graphical interface to gdb.

On Linux, the tool *Valgrind* (https://valgrind.org) can find errors in the use of memory, such as reading variables that were never initialized, even when you do not know where to look.

20.4 Exercises

The programs for these exercises are in the directory `code/checks`, and the solutions in `code/exercises/ch20`, that accompany this tutorial.

1. The program `checks/col72.f` should print 28, the sum of 1, 2 and 25. Run it. What does it print, and why? Compile it again with `-Wall`. (Look at the columns of the assignment to `total`.)

2. The program `checks/fpe.f` reads a number x and computes $y = \sqrt{x}$ and $z = 2y + 1$. Run it with the input -1, first compiled without options and then with `-g -ffpe-trap=invalid`. What is the difference? Then write a version that checks the argument and prints a message instead.

3. The program `checks/uninit.f` sums the numbers from 1 to 10 in a variable `s`, but never gives `s` an initial value. Run it compiled without options, and then with `-g -finit-real=snan -ffpe-trap=invalid`. Why is the first result so dangerous?

4. The program `checks/buggy.f` should print the mean (4.0) and the sample variance (2.0) of five numbers, but it has three errors. Find them, first by compiling with `-Wall` and then with `-fimplicit-none`, and finally by reading the code carefully. Which errors can the compiler find, and which ones only you?

Chapter 21

Fortran resources on the Web

The following resources were available when this tutorial was revised (2026). Web addresses change, so if a link does not work, search for the name.

21.1 The language

- fortran-lang.org: the home page of the Fortran community, with tutorials on modern Fortran, a list of compilers, and a catalog of Fortran packages.
- Fortran Wiki: an open wiki with articles on all aspects of Fortran, including Fortran 77.
- Fortran Discourse: a forum where you can ask questions about Fortran.
- The Fortran 77 standard: the complete text of the standard (ANSI X3.9-1978), on the site of WG5, the ISO committee for Fortran. The U.S. committee J3 also publishes the drafts of the current standards.
- Fortran 90 for the Fortran 77 Programmer: a tutorial by Bo Einarsson and Yurij Shokin on the new features of Fortran 90, written for readers who already know Fortran 77.

21.2 Compilers and tools

- gfortran documentation: the complete manual of gfortran, including all compiler options.
- MSYS2: an easy way to install gfortran on Windows (see Chapter 4).

- Other free compilers: the Intel Fortran compiler (`ifx`), LLVM Flang, and LFortran, which is still under development. Compiling your programs with more than one compiler is a good way to find errors and non-portable code.
- gdb, the GNU debugger, and Valgrind (see Chapter 20).

21.3 Numerical software

- Netlib, with BLAS, LAPACK and many other packages (Chapter 18).
- The LAPACK Users' Guide.
- GAMS, the Guide to Available Mathematical Software.
- OpenBLAS, a fast, free implementation of BLAS and LAPACK.

21.4 The original tutorial

This tutorial is based on the Fortran 77 tutorial written at Stanford University by Erik Boman, Sarah T. Whitlock and Paul H. Hargrove, which is still available at `https://web.stanford.edu/class/me200c/tutorial_77/`.